

Chat Application System Design (Like WhatsApp)

Design a real-time, scalable messaging platform supporting 1B+ users, offline delivery, group chats, and media sharing.

PAGE 1 — Title & Purpose

Topic: Design a Chat Application like WhatsApp

1-Line Purpose: Show how to architect a real-time, durable, globally distributed messaging system with delivery guarantees, at billion-user scale.

Core challenge: Messages must be delivered **at least once**, in per-conversation order, in real time when online, and reliably when offline. Idempotency and client deduplication provide an exactly-once user experience without claiming exactly-once transport.

PAGE 2 — Rapid Answer Script (2–3 minutes, speak this out loud)

"Let me walk you through how I'd design WhatsApp at a high level."

"At the core, WhatsApp is a messaging system where users send text, media, and voice to individuals or groups. The key design challenges are: real-time delivery via persistent connections, offline message buffering, media storage at scale, end-to-end encryption, and group fanout."

"I'd split the system into these major components: **Connection layer** — WebSocket servers handling persistent connections from clients. **Chat service** — validates and persists messages, then uses Redis Streams for low-latency routing; Kafka is reserved for search, analytics, CDC, and audit consumers. **Message store** — Cassandra for high-write, time-ordered messages with TTL for ephemeral data. **Presence service** — tracks online/offline status with Redis pub/sub. **Media service** — stores images/videos on S3 with CDN delivery. **Notification service** — APNs/FCM push for offline users."

"For consistency, I'd use sequence IDs per conversation for ordering. For delivery guarantees, I'd use ACK-based confirmation — the sender gets a single tick when server receives, double tick when delivered, blue tick when read."

"For scale: Cassandra handles high-volume append writes, Redis Streams handles the hot delivery path, and Kafka handles durable asynchronous integrations, WebSocket servers are stateless behind a load balancer with consistent hashing to route users to the right server."

"One key trade-off: group chat fanout — at 256 members, we fan out on write (push to each member's queue) which is simpler but burns more writes. Pull-on-read would reduce writes but add read latency."

PAGE 3 — Glossary

Term	Simple Definition	Real-World Example
WebSocket	Persistent two-way TCP connection (no HTTP overhead per message)	WhatsApp keeps your connection open so messages arrive instantly
Long Polling	Client repeatedly asks server "any new messages?"	Old Gmail before WebSocket support
Message Queue	Buffer that stores events when a consumer is busy	Redis Streams hold delivery work; Cassandra remains the durable source of truth
Fanout	Sending one message to many recipients	Group message to 256 members triggers 256 write operations
ACK (Acknowledgement)	Confirmation that something was received/processed	Single tick = server got it; double tick = device got it
Presence	Is this user online or offline right now?	Green dot in WhatsApp
TTL (Time-To-Live)	Data expires after N seconds automatically	Undelivered messages purged after 30 days in Cassandra
Consistent Hashing	Algorithm to route user to same server each time	User A always connects to WebSocket server #3
E2E Encryption	Only sender/receiver can decrypt; server sees ciphertext	WhatsApp Signal Protocol — even WhatsApp can't read your messages
CDN	Distributed cache for static content close to users	Images served from a node 20ms away, not your origin server 200ms away
CAP Theorem	Can't have Consistency + Availability + Partition Tolerance all at once	Cassandra trades consistency for availability; choose AP
Idempotency	Same operation done twice has same effect as once	Re-sending a failed message doesn't create duplicates
Sequence ID	Monotonically increasing number per conversation	Message IDs 1,2,3... tell you if you missed message 2

PAGE 4 — Decision Framework

4.1 Data Shape

Ask yourself: What does one "record" look like?

Chat message:
{

```

message_id: UUID,
conversation_id: String,
sender_id: String,
content: encrypted_bytes,
timestamp: Long,
type: TEXT | IMAGE | VIDEO | VOICE,
status: SENT | DELIVERED | READ
}

```

Pattern: Narrow, append-only, time-ordered → Cassandra / ScyllaDB wins over MySQL

4.2 Consistency vs Availability

Requirement	Choice	Why
Message ordering	Sequence IDs	Client-side reorder buffer
Delivery guarantee	At-least-once	Deduplicate on client side
Presence data	Eventually cons.	Stale presence is acceptable
User account data	Strong cons.	Can't lose registration data
Payment (if any)	Linearizable	Must not double-charge

4.3 Access Patterns

Pattern	Frequency	DB choice
Load last 50 messages (history) conv_id + timestamp)	Very High	Cassandra (range query on
Fetch undelivered messages PENDING index)	High	Cassandra (status =
Search message content index)	Low	Elasticsearch (separate
User profile lookup	Medium	Redis cache + MySQL
Group member list read-heavy)	High	Redis cache (small lists,
Media download	Very High	S3 + CloudFront CDN

4.4 Scale and Latency Requirements

Metric	Target
Message delivery latency	< 100ms (P99)
Media upload latency	< 500ms (P95)
Presence update latency	< 1 second
WebSocket reconnect	< 2 seconds

Message history load	< 200ms
Throughput	100K messages/sec peak

4.5 Team and Operational Maturity

If team has Cassandra expertise	→ Cassandra for message store
If team is SQL-native	→ PostgreSQL + partitioning (works up to ~100M users)
If team is small startup serverless)	→ Firebase Realtime DB (managed,
If team needs managed infra	→ DynamoDB + SQS + Lambda (AWS-native)

PAGE 5 — Category-by-Category Comparison Table

Category	Option A	Option B	Winner for WhatsApp-scale
Real-time transport	WebSocket	Server-Sent Events (SSE)	WebSocket (bidirectional)
Message store	Cassandra	DynamoDB	Cassandra (open-source, flexible)
Message queue / fanout	Redis Streams	Kafka	Redis Streams for hot-path latency; Kafka for replayable integrations
Presence service	Redis Pub/Sub	ZooKeeper	Redis (low latency, simple)
Media storage	S3 + CDN	Self-hosted NFS	S3 + CDN (scale, cost, reliability)
Search	Elasticsearch	PostgreSQL FTS	Elasticsearch (horizontal scale)
Push notifications	FCM / APNs	WebSocket fallback	FCM/APNs (works when app is killed)
Cache	Redis	Memcached	Redis (richer data structures)
SQL store (accounts)	PostgreSQL	MySQL	Either works; PostgreSQL for JSONB
Service mesh / routing	Nginx + Consistent Hash	HAProxy	Nginx (easy WebSocket upgrade header)

PAGE 6 — Category Deep Dives (Top 2 Technologies Each)

6.1 Real-Time Transport: WebSocket vs SSE

Difference:

- WebSocket: Full-duplex TCP. Client and server both push at any time.

- SSE: Server-to-client only. Client must use HTTP POST separately to send.

Why pick WebSocket:

- Chat is inherently bidirectional — typing indicators, message send, ACKs all go both ways.
- One connection handles send + receive.
- Better on mobile (fewer connections).

Why pick SSE:

- Simpler to implement behind HTTP/2 proxies.
- Auto-reconnects natively via EventSource browser API.
- Works with standard HTTP load balancers without sticky sessions tricks.

WebSocket vs SSE — specific scenarios:

- **WhatsApp/Telegram clone** → **WebSocket** (bidirectional, mobile-first)
- **Live sports score ticker** → **SSE** (server pushes only, no user input)
- **Collaborative doc editing (Google Docs clone)** → **WebSocket** (sends deltas both ways)
- **Notification feed (Facebook-style)** → **SSE** (server pushes, user clicks via REST)

When to choose WebSocket: Any real-time feature where user also sends data back (chat, live cursor, gaming).

When to choose SSE: Dashboards, feeds, score updates where client only reads.

Realistic examples:

1. WhatsApp: WebSocket to send message + receive ACK + receive incoming messages on same connection
2. Slack: WebSocket for typing indicators, reactions, thread updates all over one socket

6.2 Message Store: Cassandra vs DynamoDB

Difference:

- Cassandra: Open-source, self-managed or DataStax, highly tunable, excellent for time-series.
- DynamoDB: AWS-managed, serverless scaling, pay-per-request, strong single-table design.

Why pick Cassandra:

- Wide rows: store all messages in a conversation as one partition row → blazing range reads.
- No per-request cost (important at WhatsApp's 100K msg/sec).
- Better multi-datacenter replication control.
- Query flexibility with CQL (Cassandra Query Language).

Why pick DynamoDB:

- Zero ops overhead — no cluster sizing, no compaction tuning.
- Global Tables for multi-region replication out of the box.
- Integrates natively with Lambda, SQS, EventBridge.

Cassandra vs DynamoDB — specific scenarios:

- Team of 3 engineers, Series A startup → **DynamoDB** (no ops cost)
- 1B users, custom DC setup → **Cassandra** (cost at scale, control)
- Heavy time-range queries (fetch last 100 messages) → **Cassandra** (clustering key on timestamp)
- Serverless / AWS Lambda backend → **DynamoDB** (native integration)

Realistic examples:

1. Discord: Migrated from MongoDB → Cassandra for message history (announced publicly)
 2. Netflix: Uses Cassandra for time-series event data at 1M+ writes/sec
-

6.3 Message Queue / Fanout: Redis Streams vs Kafka

Difference:

- Kafka: Log-based, durable, replayable, partitioned by key. Pull-based consumers.
- RabbitMQ: Broker-based, push delivery, message deleted on ACK. Better for task queues.

Why pick Redis Streams for the hot path:

- Low-latency delivery between Chat Service and the WebSocket server that owns the recipient connection.
- Consumer groups and pending-entry tracking support retries after a worker crash.
- One stream per WebSocket server avoids creating millions of per-user queues.

Why keep Kafka:

- Durable replay for search indexing, analytics, CDC, audit, and other independent consumers.
- A slower integration path does not block the sender or online delivery path.

Why pick RabbitMQ:

- Simpler for point-to-point task distribution (e.g., send email notification jobs).
- Native priority queues and dead-letter queues out of the box.
- Easier mental model for teams new to queuing.

Redis Streams vs Kafka — specific scenarios:

- Online/group delivery to WebSocket servers → **Redis Streams** (low latency, consumer groups)
- Message history replay for a new device → **Cassandra** (source of truth, cursor query)
- Search indexing and analytics → **Kafka** (durable replay and multiple consumers)
- Simple job worker queue (resize image) → **RabbitMQ** or a managed queue (task semantics)

Realistic examples:

1. LinkedIn: Uses Kafka as the backbone for all real-time messaging and activity feeds
2. Uber: RabbitMQ for driver/rider notification task dispatch (simpler point-to-point)

6.4 Presence Service: Redis Pub/Sub vs ZooKeeper

Difference:

- Redis Pub/Sub: Lightweight, in-memory, ephemeral channels. No persistence. Fire-and-forget.
- ZooKeeper: CP system for distributed coordination, ephemeral znodes expire on disconnect.

Why pick Redis:

- Sub-millisecond pub/sub — presence changes propagate in <1ms.
- Simple: PUBLISH "user:123:status" "online" → all subscribers get it.
- Redis also doubles as your cache (don't need two systems).
- At WhatsApp scale: one Redis cluster handles billions of presence events/day.

Why pick ZooKeeper:

- Strong consistency — presence state is authoritative (no stale reads).
- Ephemeral nodes auto-expire when connection drops (built-in heartbeat).
- Better for service registry / leader election (dual-purpose).

Realistic examples:

1. WhatsApp-style: Redis HSET stores `user_id` → `{status, last_seen}`. WebSocket server publishes on connect/disconnect.
 2. LinkedIn: ZooKeeper used for service discovery; Redis used for user presence separately.
-

6.5 Media Storage: S3 + CDN vs Self-Hosted

Difference:

- S3 + CDN: Managed object store + edge cache. Pay per GB stored and transferred.
- Self-hosted (Ceph/MinIO): On-prem or cloud VMs running object storage. Full control, higher ops.

Why pick S3 + CDN:

- 99.999999999% (11 9s) durability out of the box.
- CloudFront CDN serves images from 400+ edge locations — images load fast globally.
- No capacity planning for disk; auto-scales.
- Pre-signed URLs: server never touches media bytes, clients upload/download directly.

Why pick Self-Hosted:

- Data sovereignty requirements (GDPR, banking regulations).
- Cost at extreme scale (petabytes): S3 gets expensive.
- Custom access patterns / retention policies not available in S3.

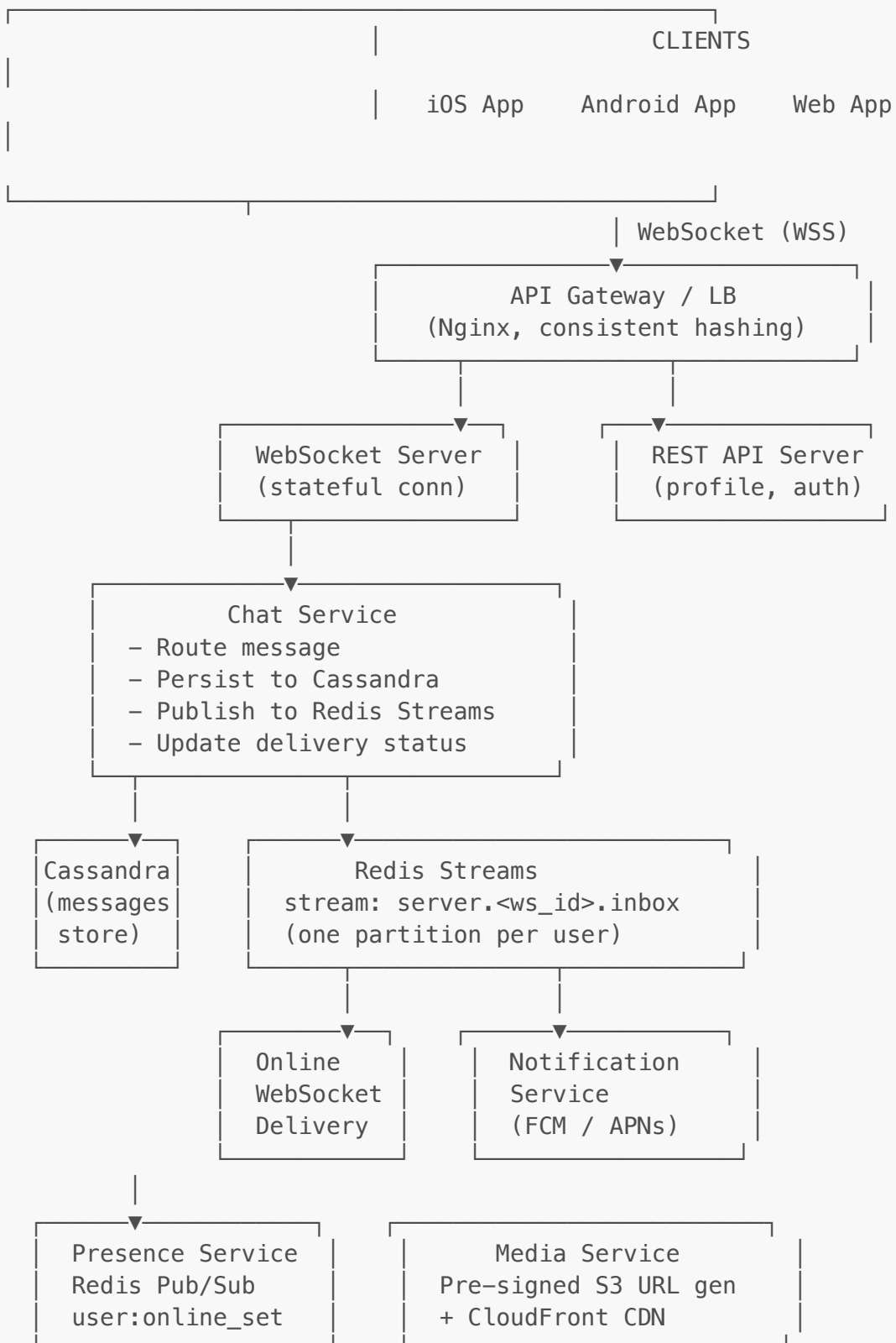
Realistic examples:

1. WhatsApp: Media uploaded directly to S3 via pre-signed URL. Thumbnail served via CloudFront.

2. A German bank: Uses MinIO on-prem for regulatory compliance — customer data cannot leave EU DCs.

PAGE 7 — End-to-End Architecture

7.1 Component View





7.2 Component, API, Event, and Configuration Responsibilities

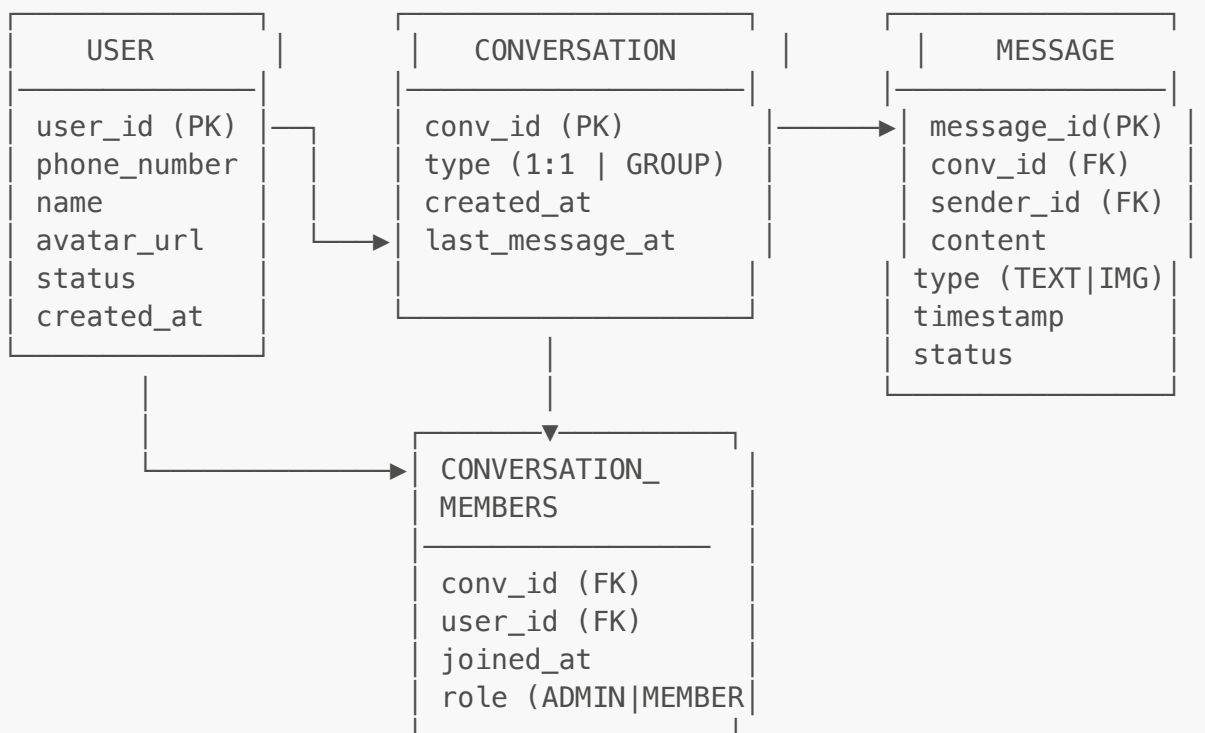
Part	Purpose	Important contract or configuration
API Gateway / load balancer	Authenticates HTTP requests, upgrades WebSockets, rate-limits, and routes traffic	WSS only; JWT validation; request and connection limits
WebSocket Gateway / Chat Service	Owns live sockets, validates commands, assigns server message IDs, and routes delivery work	Heartbeat every 30s; reconnect with jitter; never stores message payload only in process memory
User/Auth Service + PostgreSQL	Owns identity, phone uniqueness, profiles, and durable account state	Primary writes for account mutations; replicas only for non-critical reads
Group Service + PostgreSQL	Owns membership, roles, and group limits; authorizes every group operation	Use a group_members join table; do not store mutable membership only in an array
Cassandra message store	Durable append-oriented source of truth for message history and pending state	Partition by conversation/bucket; clustering by server sequence; bound partition size
Redis registry and presence	Maps user/device to WebSocket server and carries ephemeral online state	Heartbeat TTL about 60s; stale presence is acceptable; do not treat it as durable message storage
Redis Streams	Delivers work from Chat Service to the owning WebSocket server and tracks pending consumer entries	One stream per server; XREADGROUP ; retry pending entries; cap/trim retained entries
Kafka	Feeds independent, replayable consumers such as search, analytics, CDC, and audit	Topic partitioned by conversation; replication factor and retention are operational settings
Notification Service + FCM/APNs	Wakes or alerts devices that have no live WebSocket	Deduplicate notifications; notification is not proof of message delivery
Media Service + S3 + CDN	Authorizes upload, stores large blobs, and serves them efficiently	Pre-signed URL expiry about 15 minutes; size/type limits; private bucket and signed downloads

Part	Purpose	Important contract or configuration
Elasticsearch	Provides eventually consistent full-text search when plaintext indexing is allowed	Async indexing; tenant/user authorization filter; no server search for E2EE plaintext

Delivery events: **MESSAGE_ACCEPTED** (durable write), **DELIVERY_ACK** (recipient device received), **READ_ACK** (recipient displayed), **PRESENCE_CHANGED**, and **MEDIA_READY**. Events are at-least-once, carry an idempotency key, and may be retried.

Configuration to state aloud: WebSocket heartbeat/TTL, Cassandra replication and consistency level, Redis Stream consumer-group retry/claim timeout and retention, Kafka replication/retention, maximum message/media sizes, cursor page size, rate limits, JWT expiry, and regional failover/RPO targets. These values are policy knobs, not guarantees; validate them with load tests and the chosen managed-service limits.

7.3 ER View (Core Entities)



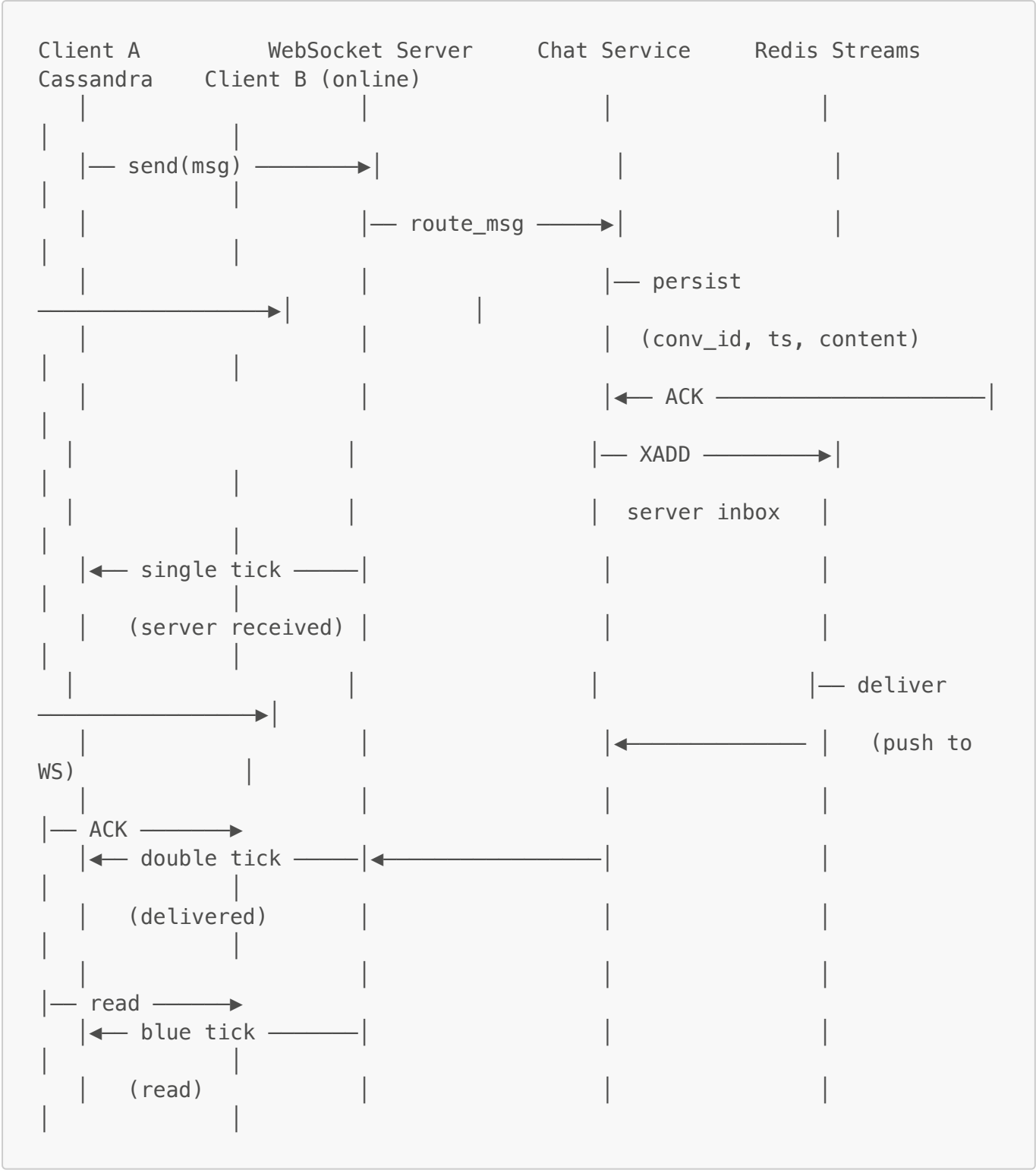
Cassandra schema for messages (partition key = conv_id, clustering = timestamp DESC):

```

PRIMARY KEY ((conv_id), timestamp, message_id)
→ Allows: SELECT * FROM messages WHERE conv_id = ? ORDER BY timestamp
DESC LIMIT 50

```

7.3 Sequence View — Critical Write Flow (Sending a Message)



Offline flow: Cassandra stores the durable message with **PENDING** delivery state. Redis Streams route work to the recipient's WebSocket server when it is online; APNs/FCM wakes an offline device. On reconnect, the Chat Service reads pending messages from Cassandra, delivers them in order, and advances delivery state after a client ACK.

PAGE 8 — Capacity Estimation

8.1 Assumptions

Total users: 2 billion registered
 Daily Active Users: 500 million
 Avg messages/day/user: 40
 Peak multiplier: 3x (assume 3x burst over average)

8.2 Message Throughput

Average messages/sec = $(500M \times 40) / 86,400$
 = $20B / 86,400$
 $\approx 231,000$ msg/sec (average)
 Peak messages/sec = $231,000 \times 3 \approx 700,000$ msg/sec

 Kafka throughput needed: $\sim 700K$ events/sec
 → 7 Kafka partitions $\times$ 100K events/sec each (with replication factor 3)

8.3 Storage

Average message size = 200 bytes (text) + 20 bytes metadata = 220 bytes
 Daily storage = 20B messages $\times$ 220 bytes = 4.4 TB/day
 Annual storage = 4.4 TB $\times$ 365 $\approx$ 1.6 PB/year (messages only)

Media:

- 10% of messages have media (2B/day)
- Avg media size = 100 KB (compressed thumbnail + original)
- Daily media storage = 2B $\times$ 100KB = 200 TB/day
- Use S3 Intelligent-Tiering to move old media to Glacier

Cassandra sizing:

- Replication factor 3 → 1.6 PB $\times$ 3 = 4.8 PB raw Cassandra storage
- Use TTL = 30 days for undelivered, 1 year for delivered (archived to S3)

8.4 WebSocket Connections

Peak concurrent connections = DAU $\times$ 80% online at peak = 400M connections
 Connections per server = 65,000 (typical Linux TCP limit per port/IP tuple)
 WebSocket servers needed = $400M / 65K \approx 6,200$ servers

Optimize: Use SO_REUSEPORT, tune ulimit, use multiple ports per machine

- With 10K connections/core, 64-core machine = 640K connections
- $400M / 640K \approx 625$ high-memory servers

8.5 Formulas to Remember

```

Messages/sec    = (DAU × msgs_per_user) / 86,400
Storage/day     = messages/day × avg_msg_size
WS servers      = peak_connections / connections_per_server
Kafka brokers   = peak_events/sec / throughput_per_broker
CDN savings     = % of reads served from edge (typically 90%+ for media)

```

PAGE 9 — Interview Scripts

9.1 Requirement Clarification Script

"Before I dive into design, let me clarify a few requirements..."

FUNCTIONAL:

- 1:1 messaging? Yes. Group messaging? Yes, up to how many members?
- Media types? Text, images, videos, voice notes, documents?
- Message history – how far back? Unlimited or rolling window?
- Read receipts? Typing indicators? Online presence?
- End-to-end encryption required?
- Message deletion? Unsend after send?

NON-FUNCTIONAL:

- What scale? Daily active users, messages per day?
- Latency: What's acceptable for delivery? (<100ms? <1s?)
- Availability: 99.99%? Can we tolerate brief downtime?
- Consistency: Can messages occasionally appear out of order client-side?
- Geography: Multi-region from day 1 or single-region MVP?

9.2 Trade-Off Script

"I see two main trade-offs here I want to call out..."

TRADE-OFF 1 – Group Fanout Strategy:

"For group messages, I can fan out on WRITE (push message to each member's inbox immediately)
or fan out on READ (store once, each member fetches when they open the chat)."

Fan-out on WRITE:

- ✓ Simple delivery: each user has their own inbox queue
- ✓ Low read latency: no need to compute group membership on read
- ✗ High write amplification: 256-member group = 256 Cassandra writes per message

Fan-out on READ:

- ✓ One write regardless of group size
- ✗ Complex: must compute group membership + fetch on every chat open
- ✗ Higher read latency

My recommendation: Fan-out on WRITE for groups ≤ 256 (WhatsApp's limit). If we had Twitter-style groups of 100K followers, fan-out on read + hybrid."

TRADE-OFF 2 – Message Ordering:

"True global ordering requires a distributed lock (expensive). Instead I'll use per-conversation sequence IDs (Cassandra clustering key = timestamp) and let clients buffer and re-sort. This gives us eventual consistency per conversation with much better performance."

9.3 Final Recommendation Script

"Here's my final recommendation..."

"For a WhatsApp-scale system with 500M DAU, I recommend:

Transport:	WebSocket (bidirectional, mobile-efficient)
Message store:	Cassandra (append-only writes, range reads on conv_id + timestamp)
Hot delivery:	Redis Streams (low latency, consumer groups)
Integrations:	Kafka (search, analytics, CDC, audit)
Presence:	Redis Pub/Sub (sub-ms latency, simple)
Media:	S3 + CloudFront CDN (11-9s durability, global edge)
Push offline:	FCM + APNs
User data:	PostgreSQL + Redis cache
Search:	Elasticsearch (async indexing via Kafka consumer)

This stack is used in production by Discord, Slack, and WhatsApp variants.

It handles 700K messages/sec peak with <100ms P99 delivery latency."

PAGE 10 — Senior Trap Questions

Q1: "How do you guarantee exactly-once delivery?"

Trap: Saying "use Kafka exactly-once semantics" without explaining client-side deduplication.

Strong answer:

"Exactly-once is hard. I'd aim for at-least-once delivery with idempotent processing. Each message has a client-generated UUID. The server deduplicates using a Redis bloom filter (or a Cassandra unique index on message_id). On the client, before rendering, check if message_id already exists in local DB. This gives practical exactly-once UX without the distributed coordination overhead of true EOS."

Q2: "What happens when a WebSocket server crashes?"

Trap: Saying "messages are lost" or "we retry" without a concrete recovery path.

Strong answer:

"WebSocket servers are stateless — they hold connections, not messages. When server crashes, clients reconnect within 2-3 seconds (exponential backoff). Messages in-flight at crash time: the sender hasn't received a server ACK yet, so it retries with the same message_id (idempotent). Pending messages for the user are fetched from Cassandra on reconnect. Kafka consumer group rebalances, another Chat Service instance picks up the partition."

Q3: "How do you handle the thundering herd on group message delivery?"

Trap: Saying "just scale up" without a concrete technique.

Strong answer:

"In a 256-member group, one message triggers 256 Redis Stream deliveries asynchronously. Each Kafka consumer (Chat Service) processes independently. But if 256 users all come online at the same time after being offline (like after a network outage), we get a storm of reconnect + fetch-pending-messages requests simultaneously. I'd use: (1) Jitter on reconnect delay, (2) Rate limit per user's history fetch, (3) Cassandra replicas and bounded reads to handle the burst. Long-term: circuit breaker in Chat Service to shed load gracefully."

Q4: "How does end-to-end encryption work with group chats?"

Trap: "Just encrypt the message" without discussing key distribution.

Strong answer:

"WhatsApp uses the Signal Protocol. For 1:1: each device has a public/private key pair. Sender encrypts with receiver's public key. Server never sees plaintext. For groups: WhatsApp uses a 'Sender Key' per group per device. When you join a group, your device receives sender keys of all current members via 1:1 encrypted messages. You then encrypt group messages with your sender key. This means group message encryption is $O(1)$ per message (not $O(n)$). The trade-off: new members can't read historical messages (forward secrecy by design)."

Q5: "How would you design the typing indicator feature?"

Trap: "Store it in the database" (catastrophically wrong — you'd write/delete millions of rows/sec).

Strong answer:

"Typing indicators are ephemeral, high-frequency, and low-durability — perfect for Redis or pure WebSocket signaling. When User A starts typing: A's WebSocket server receives a TYPING_START event. It publishes to Redis channel 'conv:123:events'. B's WebSocket server is subscribed to that channel. B's server pushes typing indicator to B's client. After 3 seconds of no new

TYPING_START, client-side timer fires TYPING_STOP. We NEVER persist this to Cassandra. Rate limit: client can only send TYPING_START once every 2 seconds to prevent flooding."

Q6: "How do you preserve ordering when two devices send concurrently?"

"Ordering is scoped to a conversation, not global. The Chat Service assigns a monotonic conversation sequence through a single logical writer or sequencer partition. Cassandra stores that sequence as the clustering key, and clients buffer gaps briefly before requesting a resync. Timestamps alone are not an ordering guarantee."

Q7: "What if the delivery ACK is lost?"

"The client retries the ACK with the same message ID. The server applies the state transition idempotently, so **PENDING** → **DELIVERED** is harmless when repeated. ACK loss must not cause a second message or a second notification."

Q8: "Why is Redis Streams not the source of truth?"

"Redis is the low-latency delivery work queue, not the durable history authority. Cassandra receives the message before the sender success ACK. A Redis outage pauses live delivery, but reconnect reconciliation reads pending messages from Cassandra."

Q9: "How do you avoid duplicate group delivery?"

"Use a stable message ID plus a per-recipient delivery record or inbox key. Fanout retries may run more than once, but the recipient applies each message once and delivery state advances monotonically."

Q10: "What happens when a group has millions of members?"

"Do not synchronously fan out to every member. Store one group message, use fanout-on-read or a hybrid, batch delivery, and apply backpressure. The 256-member write-fanout choice is a product constraint, not a universal rule."

Q11: "How do you handle a hot conversation?"

"Bucket the conversation partition by time or sequence range, preserve a sequencer for order, and avoid one unbounded Cassandra partition. Rate limits and per-conversation backpressure protect the rest of the system."

Q12: "How does E2EE change search, moderation, and notifications?"

"The server stores ciphertext and cannot perform plaintext search or moderation. Search is client-side or opt-in encrypted indexing; push notifications use generic text or client-generated previews. Metadata minimization and device key management become first-class requirements."

Q13: "How do you make delete-for-everyone reliable?"

"Persist a deletion tombstone, emit an idempotent delete event, and have clients apply it even after reconnect. Tombstones need retention long enough to prevent an old device from

resurrecting deleted content."

Q14: "What are your failure and recovery objectives?"

"Define RPO and RTO per data class. Message writes use replicated Cassandra across failure domains; Redis streams can be rebuilt from Cassandra; clients reconnect with jitter. Presence can be lost and reconstructed, while accepted messages cannot."

Q15: "How do you prove this design works?"

"Load test connection counts, fanout, hot partitions, reconnect storms, stream redelivery, Cassandra repair, and regional failover. Measure P99 delivery latency, duplicate rate, pending age, ACK lag, stream depth, and message loss, not only average throughput."

PAGE 11 — What Not to Say

- ❌ "I'd use MySQL for messages"
 - MySQL can't handle 700K writes/sec without heavy sharding. Cassandra is purpose-built for this.
- ❌ "I'd use HTTP polling for real-time"
 - Every poll burns a full HTTP round-trip. At 500M users, this crushes your servers.
 - Say: "WebSocket for real-time, HTTP fallback only for legacy clients."
- ❌ "I'd store typing indicators in the database"
 - Typing events fire every keystroke – that's 10+ events/sec per active user.
 - Cassandra would explode. Use Redis ephemeral pub/sub only.
- ❌ "Messages are strongly consistent globally"
 - Impossible at this scale without massive latency. Say: "eventually consistent per conversation, with client-side sequence ID reordering buffer."
- ❌ "One big PostgreSQL database handles everything"
 - True for 10K users. At 500M users, PostgreSQL becomes your single point of failure.
 - Say: "PostgreSQL for user accounts (strong schema), Cassandra for messages (scale)."
- ❌ "I'll add full-text search in Cassandra"
 - Cassandra has no full-text index. Route search to Elasticsearch via async Kafka consumer.
- ❌ "Group messages fan out synchronously in the request path"
 - Synchronous fanout to 256 members in the HTTP request thread = 256 blocking writes,
 - user waits for all to complete. Put fanout on Redis Streams – return the accepted ACK immediately.
- ❌ "I don't need a CDN for media"

→ Without CDN, every image download hits your S3 origin. At 2B media/day, S3 egress costs alone would be \$100M+/year. CDN brings 90%+ cache hit rate.

PAGE 12 — Key Numbers to Memorize

SCALE

WhatsApp users:	2B registered, 500M DAU
Messages/day:	100 billion (reported)
Peak messages/sec:	~1.4M (100B / 86400 × 1.2 burst)
Group size limit:	256 members (WhatsApp), 100K (Telegram channels)

LATENCY TARGETS

WebSocket message delivery:	< 100ms P99 (online recipient)
Offline delivery:	Seconds to minutes (depends on reconnect)
Media upload:	< 500ms for compressed thumbnail
Presence update lag:	< 1 second

CASSANDRA

Write throughput/node:	~10K–50K writes/sec
Read throughput/node:	~10K–30K reads/sec
Replication factor:	3 (1 cross-region for DR)
Recommended partition size:	< 100MB (avoid hot partitions)

KAFKA

Throughput/broker:	100–500 MB/sec
Message retention default:	7 days (enough for long offline periods)
Partition recommendation:	1 partition per "inbox" or conversation bucket

WEBSOCKET

Connections per server:	10K–100K (depending on memory)
Heartbeat interval:	30 seconds
Reconnect timeout:	2–5 seconds with exponential backoff

REDIS

Read/write latency:	< 1ms
Typical cluster size:	3 master + 3 replica for HA
Pub/Sub max:	Millions of channels, <1ms delivery

S3 + CDN

S3 durability:	99.999999999% (11 nines)
CDN cache hit target:	> 90% for media
Pre-signed URL TTL:	15 minutes (for direct upload)

ESTIMATED INFRA (500M DAU)

WebSocket servers:	~625 (64-core, 256GB RAM)
Chat service instances:	~100 (stateless, horizontal scale)

Cassandra nodes:	~200 (for 5PB raw storage)
Kafka brokers:	~50 (at 1M events/sec peak)

PAGE 13 — Whiteboard Draw Order

Follow this sequence when drawing on the whiteboard:

STEP 1 – Clients (30 seconds)

Draw: iOS | Android | Web boxes at the top

Say: "Users connect via persistent WebSocket"

STEP 2 – Entry Layer (30 seconds)

Draw: Load Balancer → WebSocket Server pool

Say: "Consistent hashing routes each user to the same WS server"

STEP 3 – Core Services (1 minute)

Draw: Chat Service box

Say: "Chat Service does three things: persist, route, notify"

Draw arrows to: Cassandra, Kafka, Presence/Redis

STEP 4 – Persistence (45 seconds)

Draw: Cassandra cluster

Say: "Messages go here – partitioned by conversation, clustered by timestamp"

STEP 5 – Async Fanout (45 seconds)

Draw: Kafka → Chat Service → Online WS delivery + Notification Service

Say: "Kafka decouples write from delivery – sender gets ACK before fanout completes"

STEP 6 – Offline Path (30 seconds)

Draw: Notification Service → FCM/APNs → Mobile

Say: "Offline users get push notification; fetch messages on reconnect from Cassandra"

STEP 7 – Media (30 seconds)

Draw: S3 + CloudFront off to the side

Say: "Media never goes through Chat Service – client uploads directly via pre-signed URL"

STEP 8 – Zoom In on Sequence (2 minutes)

Draw the send-message sequence flow (sender → server → Cassandra → Kafka → receiver)

Label the ticks: ✓ server got it, ✓✓ delivered, ✓✓ (blue) read

STEP 9 – Capacity numbers (1 minute)

Write in corner: "700K msg/sec peak | 4.4TB/day | 625 WS servers | 200 Cassandra nodes"

PAGE 14 — How to Adapt This Guide for Any Company

Fintech (e.g., Revolut, PayPal chat with transactions)

Extra requirements:

- Messages may trigger payment flows → need SAGA for distributed transactions
- Compliance: message logs retained 7 years → S3 Glacier + encryption at rest
- Strong consistency for payment ACKs (no "eventual" on money movement)
- Audit log: all messages immutably archived to WORM storage

Key differences from WhatsApp design:

- Add a Payment Service that participates in message ACK flow
- Replace Cassandra TTL with immutable archival to S3
- Add KMS (Key Management Service) for encryption key rotation

E-Commerce (e.g., Amazon seller-buyer chat)

Extra requirements:

- Messages contextualized to an Order (attach order_id to conversation)
- Bot integration (automated responses: "your order ships Nov 5")
- Seller performance metrics (response time SLA)

Key differences:

- Conversation metadata links to Order entity in PostgreSQL
- Add a Bot Service that subscribes to Kafka and injects automated replies
- Analytics pipeline from Kafka → Spark → dashboard for response time SLA

Social (e.g., Instagram DMs)

Extra requirements:

- Rich media: stories, reels, polls shared in DMs
- Message reactions (emoji reactions)
- Vanishing messages (self-destruct after view)

Key differences:

- Reactions stored as a separate Cassandra table (low write, appended fast)
- Vanishing: TTL set to 0 after recipient reads; Redis pub/sub triggers deletion
- Media references use CDN, but content-addressed (deduplicate identical media)

SaaS (e.g., Intercom, Zendesk live chat)

Extra requirements:

- Multiple agents handle same inbox (round-robin or skill-based routing)
- Conversation history needs CRM integration (Salesforce, HubSpot)
- SLA tracking: respond within 5 minutes or escalate

Key differences:

- Add Routing Service (selects agent based on availability/skills)
- Kafka consumer pushes to CRM webhook on conversation events
- Dedicated SLA Monitor service reads from Kafka, fires alerts on breach

PAGE 15 — Common Follow-Up Questions

Q: "How would you handle message search?"

"Search is a separate concern from delivery. I'd stream all messages from Kafka to an Elasticsearch consumer (async, no impact on delivery path). Search is eventually indexed (~1-2 second lag). For privacy: index only the requesting user's messages in their shard. For E2EE chats: only client-side search is possible since server never sees plaintext."

Q: "How do you scale WebSocket servers horizontally?"

"WebSocket servers hold persistent connections — they're stateful in the connection sense. I use consistent hashing on user_id to route the same user to the same server. When a server dies, clients reconnect; consistent hash ring reassigns them. Message routing between Chat Service instances: when Chat Service needs to deliver to User B who's connected to WS Server #3, it either (a) publishes to a Redis channel that WS Server #3 is subscribed to, or (b) uses a service registry to direct-call WS Server #3."

Q: "How do you handle message ordering in a distributed system?"

"Perfect global ordering requires a single writer (bottleneck). I don't need it. I need per-conversation ordering. Cassandra's clustering key on timestamp gives me server-side ordering per partition (conversation). For messages within the same millisecond, I use a tie-breaker (message_id UUID). On the client, a 200ms buffer window reorders any out-of-order arrivals before rendering. This is identical to how WhatsApp/iMessage work."

Q: "How would you implement 'delete for everyone'?"

"Store a 'deleted' flag on the message row (soft delete). When sender deletes: (1) Update Cassandra row to set deleted=true (or add to a tombstone table), (2) Publish DELETE_EVENT to Kafka, (3) Kafka consumer delivers DELETE_EVENT to all group members' WebSocket connections, (4) Clients remove from local DB + UI. Hard limit: WhatsApp only allows delete within 60 hours. After that, server no longer delivers delete events (the old message rows are already expired via TTL)."

Q: "How do you handle multi-device support (phone + laptop logged in together)?"

"Each device registers separately and has its own WebSocket connection. When a message arrives, Chat Service fans out to ALL active sessions for that user. Kafka topic is partitioned by user_id, and all devices for that user are subscribed to the same partition. Read receipts sync across devices: any device marking 'read' updates the status, and a READ_SYNC event is sent to all other devices. Message history: new device fetches last N messages from Cassandra on first login."

Q: "What's your disaster recovery strategy?"

"Multi-region active-active with Cassandra's built-in multi-DC replication (RF=3, one DC per region). Kafka uses MirrorMaker 2 to replicate topics cross-region. Redis: Redis Cluster with cross-region sentinel. RTO target: <30 seconds (client reconnects). RPO target: 0 for messages (Cassandra replication is synchronous within quorum). CDN media: S3 Cross-Region Replication for durability; CDN serves from nearest healthy region."

PAGE 16 — Final Quick-Revision Cheat Sheet (One Page)

WHATSAPP SYSTEM DESIGN – QUICK REVISION CARD

CORE COMPONENTS

```

Client → LB → WebSocket Server → Chat Service → Cassandra
                                   ↓
                               Redis Streams → Online Deliver / FCM
                                   ↓
Redis (presence) | S3+CDN (media) | PostgreSQL (users)
  
```

KEY TECH CHOICES (with WHY)

```

WebSocket    → bidirectional, mobile-efficient (not SSE/polling)
Cassandra    → write-heavy, time-ordered, wide-row partitioning
Redis Streams → low-latency delivery work between chat servers
Kafka        → replayable integrations: search, analytics, audit
Redis PubSub → sub-ms presence updates, ephemeral (not DB)
S3 + CDN     → 11-9s durability, 90%+ cache hit, pre-signed upload
FCM/APNs     → push to killed app, works when WS is disconnected
  
```

CASSANDRA SCHEMA (most asked)

PRIMARY KEY ((conv_id), timestamp DESC, message_id)

→ Fetch last 50 msgs: WHERE conv_id=? ORDER BY timestamp DESC LIMIT 50

DELIVERY GUARANTEE

At-least-once + idempotency via message_id UUID deduplication

Single tick: server ACK | Double tick: device ACK | Blue: read ACK

GROUP FANOUT

≤256 members → fan-out on WRITE (push to each server inbox via Redis)

>10K members → fan-out on READ + hybrid (store once, fanout lazily)

CAPACITY (500M DAU)

~700K msg/sec peak | 4.4 TB messages/day | 200TB media/day

~625 WS servers (64-core) | ~200 Cassandra nodes | ~50 Kafka brokers

TOP TRADE-OFFS TO MENTION

1. Fan-out on write vs read (256 writes vs 1 write with complex reads)
2. Strong ordering vs performance (sequence IDs + client buffer)
3. E2EE vs server-side features (can't do server search on E2EE)
4. Presence accuracy vs scale (stale by 1s is fine, real-time is costly)

WHAT NOT TO SAY

x MySQL for messages x HTTP polling x Typing in DB
x Synchronous group fanout x "Global strong consistency"

INTERVIEW LINE TO OPEN WITH

```
"WhatsApp's core challenge is delivering messages at least once,  
in real-time when online, and reliably when offline – at 500M DAU."
```

PAGE 17 — API Design (Missing from Original Guide)

Interview line: "Let me walk through the REST and WebSocket endpoints before jumping into the architecture."

17.1 User Onboarding (HTTP REST)

```
POST /api/v1/register  
  Body: { phone_number, name, device_token }  
  Response: { user_id, jwt_token }  
  
POST /api/v1/login  
  Body: { phone_number, otp }  
  Response: { user_id, jwt_token, session_id }
```

17.2 Chat History (HTTP REST — lazy load / pagination)

```
GET /api/v1/chats?user_id={uid}&limit=20&offset=0  
  → Returns list of conversations (preview of last message per chat)  
  
GET /api/v1/chats/{chat_id}/messages?limit=50&before_timestamp={ts}  
  → Returns paginated messages for a 1:1 chat (scroll-up = fetch older)  
  → Note: CURSOR-BASED pagination using timestamp, NOT page numbers  
  → Why: page numbers drift as new messages arrive; timestamp cursor stays  
  stable
```

17.3 Group Management (HTTP REST)

```
POST /api/v1/groups  
  Body: { group_name, member_ids[], created_by }  
  Response: { group_id }  
  
POST /api/v1/groups/{group_id}/members  
  Body: { user_id }  
  
DELETE /api/v1/groups/{group_id}/members/{user_id}
```

```
GET /api/v1/groups/{group_id}/messages?limit=50&before_timestamp={ts}
→ Same cursor-based pagination as 1:1 chat history
```

17.4 Real-Time Messaging (WebSocket)

```
WS wss://chat.example.com/ws?token={jwt}
```

Client → Server events:

```
{ type: "SEND",      chat_id, message_id (client UUID), content, msg_type }
{ type: "TYPING",   chat_id }
{ type: "READ_ACK", message_id }
```

Server → Client events:

```
{ type: "MESSAGE",      message payload }
{ type: "DELIVERY_ACK", message_id, status: "DELIVERED" }
{ type: "READ_ACK",     message_id, status: "READ" }
{ type: "TYPING",       chat_id, user_id }
{ type: "PRESENCE",     user_id, status: "online"|"offline", last_seen }
```

Key insight for interviewers: Delivery receipts and read receipts are NOT REST calls. They flow over the same WebSocket connection that was already open — no extra HTTP round-trip needed.

17.5 Media Upload (HTTP — NOT WebSocket)

```
POST /api/v1/media/upload
```

```
Body: multipart/form-data { file }
```

```
Response: { media_url (S3 pre-signed URL), media_id }
```

→ After upload succeeds, client sends media_url via WebSocket SEND event
 → Media NEVER goes through the WebSocket connection (too large, wrong protocol)

PAGE 18 — Connection Type Comparison: Why WebSocket?

Interview line: "Before I pick WebSocket, let me quickly show why the alternatives all fail for chat."

18.1 The Four Options

Type	Direction	How it works
HTTP (REST)	Client → Server	Client sends request, server

responds.	one shot	Connection closes after response.
Long Polling	Client → Server	Client sends request, server HOLDS it
	(delayed)	open until data is ready (max ~30s).
immediately.		Then closes. Client re-opens
SSE	Server → Client	Server pushes events to client.
(Server-Sent HTTP. Events) (EventSource).	one direction	Client can only receive, sends via Native browser reconnect
WebSocket	BOTH directions	One persistent connection. Both sides
	simultaneously	push any time. Full-duplex TCP.

18.2 Why Each Fails for Chat

HTTP (plain REST):

Problem: To receive messages, client must POLL – send GET every N seconds.

At 500M users polling every 1s → 500M requests/sec just to check for replies.

99% of those requests return empty. Pure waste.

❌ Not viable.

Long Polling:

Problem: You don't know WHEN your friend will reply.

Opening a 30s hold connection hoping for a reply → closes → reopens → closes.

Each reconnect = new TCP handshake = overhead.

At scale, millions of half-open connections consume server threads.

❌ Works for simple notifications, not for real-time bidirectional chat.

SSE (Server-Sent Events):

Problem: Server can push to client, but client must send via a SEPARATE HTTP call.

To send a message: 1 HTTP POST. To receive reply: SSE event fires, then 1 HTTP GET.

Minimum 2–3 connections per user. Also: SSE is HTTP/1.1, not efficient for mobile.

✗ Acceptable for feed/notification, not efficient enough for chat at scale.

WebSocket:

- ✓ One persistent connection handles BOTH send and receive.
- ✓ After the initial HTTP handshake, no more HTTP headers per message (much lighter).
- ✓ Bidirectional: ACKs, typing indicators, messages – all on one socket.
- ✓ Works efficiently on mobile (battery + bandwidth friendly).

18.3 How WebSocket Connection is Established

Step 1: Client sends HTTP request with special headers:

```
GET /ws HTTP/1.1
Upgrade: websocket
Connection: Upgrade
```

Step 2: Server validates, responds:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
```

Step 3: TCP connection is now a persistent WebSocket.

→ From this point: no HTTP headers per message. Pure binary/text frames.

Key fact: ALL WebSocket connections start as HTTP.

The HTTP handshake upgrades to WS. This is why WS works through HTTP load balancers.

PAGE 19 — WebSocket Registry + Sticky Sessions + TTL

Interview line: "The WebSocket registry is the brain of message routing — without it, you can't deliver a message to the right server."

19.1 The Problem: Multiple Chat Servers

Chat Server 1

User A connected
User C connected

Chat Server 2

User B connected
User D connected

If User A sends a message to User B:

- Chat Server 1 receives it
- But User B is connected to Chat Server 2!
- How does Chat Server 1 know WHERE User B is?
- Answer: WebSocket Registry (a shared Redis hash map)

19.2 WebSocket Registry Structure (Redis)

Redis Hash: "ws_registry"

Key = user_id

Value = { server_id, connection_id, last_heartbeat }

Example entries:

user:1001 → { server: "ws-server-3", conn_id: "conn_abc", heartbeat: 1721234567 }

user:1002 → { server: "ws-server-1", conn_id: "conn_xyz", heartbeat: 1721234590 }

user:1003 → { server: "ws-server-3", conn_id: "conn_def", heartbeat: 1721234512 }

Operations:

On connect: HSET ws_registry user:1001 { server, conn_id, heartbeat }

On disconnect: HDEL ws_registry user:1001

On heartbeat: HSET ws_registry user:1001 heartbeat=<now> (refresh TTL)

On lookup: HGET ws_registry user:1001 → which server to route to

19.3 Message Routing Using the Registry

User A (on ws-server-1) sends message to User B:

Step 1: Chat Service receives message via ws-server-1

Step 2: Chat Service looks up ws_registry for User B

→ Found: User B is on ws-server-2

Step 3: Chat Service publishes to Redis channel for ws-server-2:

PUBLISH "server:ws-server-2:inbox" { message for User B }

Step 4: ws-server-2 is subscribed to its own inbox channel

→ Receives the message, pushes to User B's open socket

Step 5: User B's client sends READ_ACK back over same socket

→ ws-server-2 publishes ACK to Redis → ws-server-1 → User A gets blue tick

19.4 Sticky Sessions — Why They Matter

"Sticky" means: the same user ALWAYS routes to the same WebSocket server.

Why it's needed:

- WebSocket is a stateful connection (tied to a specific server process)
- If the load balancer randomly picks a server each time, your socket breaks
- Reconnects must land on the SAME server to resume the connection

How it's implemented:

- WebSocket Gateway uses CONSISTENT HASHING on user_id
- user_id hash → always maps to the same ws-server

- Even if you add more servers (ring rebalances), same user stays sticky

Fallback: If sticky server dies → client reconnects → consistent hash picks new server → new entry in ws_registry. Old entry expires via TTL.

19.5 TTL on WebSocket Connections

Problem: At 1B users, you can't hold 1B open WebSocket connections simultaneously.

- WebSocket connections are heavyweight (memory, file descriptors)
- Most users open WhatsApp briefly, then background the app

Solution: TTL (Time-to-Live) on registry entries

Each ws_registry entry has TTL = 60 seconds

Client sends a heartbeat ping every 30 seconds to refresh the TTL

If no heartbeat for 60s → entry expires → user treated as offline

On expiry:

1. Registry entry deleted
2. User Management Service notified (via Redis keypace event)
3. last_seen updated in User DB
4. Any subsequent messages for this user go to Cassandra (offline queue)

Why 60s TTL? Balance between:

- ✓ Quick detection of dead connections (don't hold resources)
- ✓ Tolerates brief network blips without marking user offline
- ✗ Too short (5s) → false offline triggers on 4G handoffs
- ✗ Too long (10m) → waste resources on dead connections

PAGE 20 — Redis Streams vs Kafka: Which for Real-Time Chat?

Interview line: "Both Redis Streams and Kafka can do pub/sub fanout — but for chat, latency beats throughput, so I lean toward Redis Streams for the hot path."

20.1 Core Difference

Attribute	Redis Streams	Kafka
Latency	< 1ms (in-memory)	2–10ms (disk-based)
Persistence	In-memory (+ optional)	Durable disk log (days)

	AOF/RDB snapshots)	
Throughput	High (millions/sec)	Very high (millions/sec)
Replay	Limited (stream history)	Full replay (weeks/months)
Ops complexity	Simple (part of Redis)	Needs ZooKeeper/KRaft, JVM
Consumer groups	Yes (XREADGROUP)	Yes (consumer groups)
Best for	Real-time routing, low-latency delivery	Event sourcing, analytics, audit logs, CDC pipelines

20.2 Recommended Split for WhatsApp

Use REDIS STREAMS for:

- Live message delivery to online users (latency critical: <1ms)
- Typing indicator fanout
- Presence event broadcasting
- Delivery/read ACK routing

Channel model: one stream per chat-server (not per user – too many streams)

```
XADD "stream:ws-server-3" * {message payload for user B}
```

Use KAFKA for:

- Async indexing to Elasticsearch (search index – delay is fine)
- Feeding analytics pipeline (how many messages/day per group)
- Audit/compliance log (immutable replay)
- CDC from Cassandra to data warehouse

Topic model: one topic "messages" with partitions by conversation_id

Why not Kafka for real-time delivery?

Kafka adds 2–10ms just for broker acknowledgement.

At 100K messages/sec, that's 200–1000 seconds of aggregate added latency.

Redis Streams at <1ms keeps P99 delivery under 100ms.

20.3 Channel Design in Redis Streams

Option A – One stream per user:

Stream key: "user:1001:inbox"

Each chat server reads streams for users it hosts

Problem: At 500M DAU, 500M stream keys. Memory expensive.

Option B – One stream per chat server (recommended):

Stream key: "server:ws-server-3:inbox"

All messages destined for users on ws-server-3 go here
 ws-server-3 reads its stream and dispatches to individual sockets
 Only N streams where N = number of chat servers (~625)
 ✅ Much more memory efficient. ✅ Simple fan-in/out at server level.

PAGE 21 — Media Upload: Two-Step Flow

Interview line: "Media is never sent through WebSocket — it's too large. The correct pattern is: HTTP upload first, then send the URL over WebSocket."

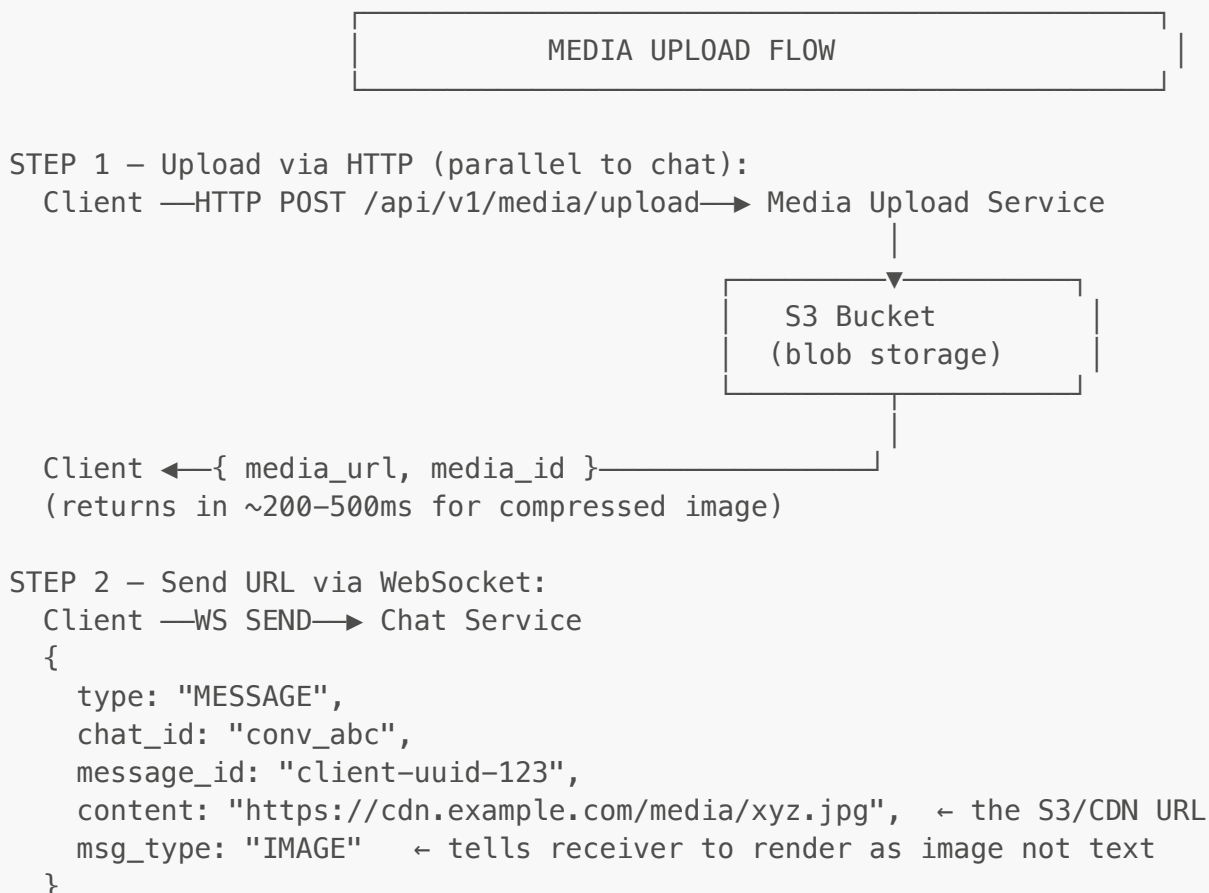
21.1 Why Media Can't Go Through WebSocket

Problem with sending binary media via WebSocket:

- A 5MB video as a WebSocket frame blocks the socket for all other messages
- No retry/resume for partially transferred files
- Chat server becomes a bottleneck processing every byte of every video
- S3 is specifically designed for large binary blobs; WebSocket is not

Solution: Two-step upload

21.2 The Two-Step Upload Flow



STEP 3 – Receiver loads media via CDN:

Receiver's client gets the message with the media URL
 Client fetches image from CloudFront CDN (NOT from S3 directly)
 CDN cache hit → ~20ms. CDN miss → CDN fetches from S3 → ~200ms.

21.3 Pre-Signed URL Pattern (Optimization)

Instead of routing all uploads through your server:

Client —GET /api/v1/media/presign→ Media Service
 Media Service —generates→ S3 Pre-signed PUT URL (valid 15 min)
 Media Service returns pre-signed URL to client

Client —HTTP PUT directly to S3 Pre-signed URL→ S3
 (client uploads DIRECTLY to S3, bypassing your servers entirely)

After S3 confirms upload, client sends media_id via WebSocket

Benefits:

- ✓ Your servers never touch media bytes → massive bandwidth savings
- ✓ S3 handles large file transfers natively (multipart, resume)
- ✓ Client → S3 at full speed (no bottleneck through your API layer)

21.4 CDN for Media Delivery

Without CDN:

500M users, each loading 5 images/day = 2.5B S3 GET requests/day
 S3 egress cost + latency for users far from S3 region

With CDN (CloudFront):

First request for image.jpg → CDN miss → CDN fetches from S3 → caches at edge

All subsequent requests → CDN hit → served from edge PoP (< 20ms)

Cache hit ratio for shared media (stickers, GIFs): ~95%+

Cache hit ratio for personal photos (unique): ~40–60%

Memory anchor: CDN = geographic copy of your S3 bucket, close to users.

PAGE 22 — Gaps from Video: Local Cache, last_seen, Offline Reconnect

22.1 Mobile-Side Local Cache (WhatsApp's Server Deletion Trick)

Interview line: "WhatsApp claims they don't store your messages on their servers. Here's how that's architecturally possible."

The claim: WhatsApp does not persist messages server-side after delivery.

How it works:

1. Message arrives → stored in Cassandra (pending delivery)
2. Message delivered to recipient's device → device stores in local SQLite DB
3. Server DELETES the row from Cassandra (delivered = no longer needed server-side)
4. Chat history loaded from LOCAL device storage, not from server

Why this is smart:

- ✓ Reduces Cassandra storage by ~90% (only undelivered messages persist)
- ✓ Chat history loads instantly (no network call)
- ✓ Privacy: server can't be compelled to produce message contents

Limitations:

- ✗ New device login: chat history from OLD device is lost (unless backup enabled)
- ✗ WhatsApp backup (Google Drive/iCloud) is separate opt-in mechanism
- ✗ E2EE means server can't re-sync history even if it wanted to

What stays on server:

- Undelivered messages (offline users) – deleted on delivery
- Message metadata (timestamps, chat IDs) – for delivery status only
- Media thumbnails cached briefly for delivery

Interview trap: Don't say "WhatsApp has a huge message DB."
They deliberately minimize what's stored server-side.

22.2 User Management Service — last_seen Updates

Challenge: How do you update "last seen 2 minutes ago" in the User DB?

Naive approach (wrong): Poll the WebSocket registry every second.

→ 500M lookups/sec just to check for idle users. Kills Redis.

Correct approach: Redis Keyspace Notifications + User Management Service

Step 1: Enable Redis keyspace events:

```
CONFIG SET notify-keyspace-events "Kx"
(notify on key EXPIRY events)
```

Step 2: When a WebSocket registry entry EXPIRES (TTL hits 0):

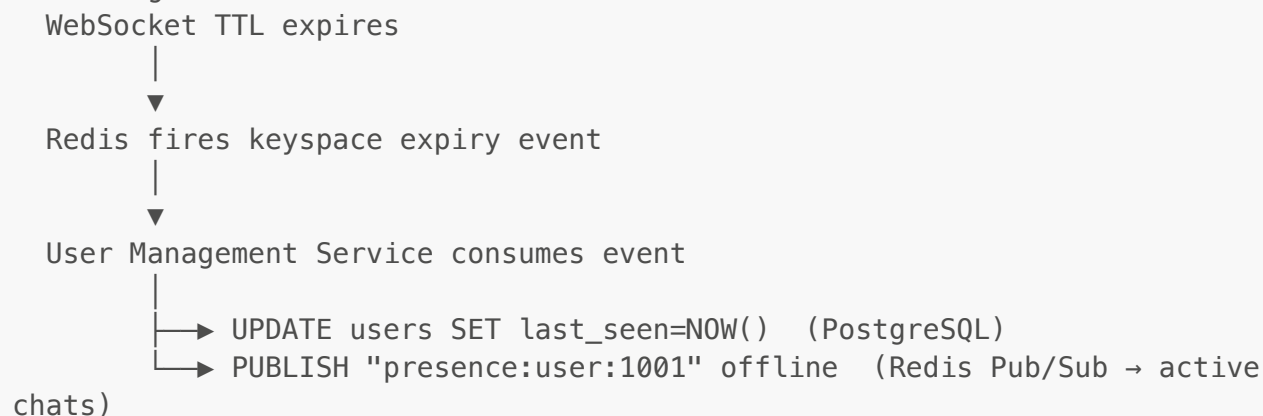
Redis publishes event: "__keyevent@0__:expired" →
"ws_registry:user:1001"

Step 3: User Management Service subscribes to these expiry events:

On event received → update User DB:
UPDATE users SET status='offline', last_seen=NOW() WHERE
user_id=1001

Step 4: Other users who have a chat open with user 1001:
 Presence Service detects the change → pushes PRESENCE event via WebSocket
 "user:1001 is now offline, last seen 14:32"

Flow diagram:



22.3 Offline User Reconnect Flow (Complete)

User was offline (phone dead/no internet). Now comes back online.

Step 1 – Establish WebSocket:

Client → HTTP upgrade request → WebSocket Gateway
 Gateway → creates WebSocket connection on Chat Server
 Chat Server → HSET ws_registry user:1001 {server, conn_id}

Step 2 – Fetch undelivered messages:

Chat Server → queries Cassandra:
 SELECT * FROM messages
 WHERE receiver_id = 1001
 AND delivery_status = 'PENDING'
 ORDER BY timestamp ASC

Step 3 – Deliver messages:

Chat Server → pushes all pending messages via WebSocket to client
 Client renders messages in order

Step 4 – Send DELIVERY_ACK:

Client → sends DELIVERY_ACK for each message over WebSocket
 Chat Server → updates Cassandra: delivery_status = 'DELIVERED'
 Chat Server → notifies original sender's Chat Server via Redis Streams
 Original sender receives double-tick ✓✓

Step 5 – Send READ_ACK (when user opens the chat):

Client → sends READ_ACK for opened chat
 Chat Server → updates Cassandra: delivery_status = 'READ'
 Original sender receives blue tick ✓✓ (blue)

Key design point: Steps 2-5 all happen automatically on reconnect without

any user action. That's why your messages appear instantly when WhatsApp comes back online after being offline.

22.4 Group Message: Why Individual Fanout (Not Broadcast)

Question interviewers ask: "Why not just broadcast to a group channel?"

Naive approach:

```
PUBLISH "group:123:messages" {message}
```

All group members subscribed → all receive it simultaneously

Why this is wrong for WhatsApp-style groups:

- You cannot track per-member delivery status with a broadcast
- WhatsApp shows: "Delivered to 245/256 members, Read by 189/256"
- That level of granularity requires sending to EACH member individually

Correct approach: Individual fanout per group member

1. Chat Service calls Group Service: GET /groups/123/members
→ Returns [user_1, user_2, ..., user_256]
2. For EACH user_id:
 - a. Check ws_registry: online or offline?
 - b. If online → push to their chat server's Redis stream
 - c. If offline → persist to Cassandra with delivery_status='PENDING'
→ send FCM/APNs push notification
3. Each member's DELIVERY_ACK updates the per-member status row

Schema for group message delivery tracking:

```
TABLE group_message_delivery (
  message_id  UUID,
  group_id    VARCHAR,
  user_id     VARCHAR,
  status      ENUM('PENDING', 'DELIVERED', 'READ'),
  delivered_at TIMESTAMP,
  read_at     TIMESTAMP,
  PRIMARY KEY ((message_id), user_id)
)
```

Trade-off: 256 members × message = 256 Cassandra writes per group message.
At 1000 group messages/sec → 256K writes/sec just for group delivery tracking.

Cassandra handles this comfortably (50K+ writes/node × multiple nodes).

22.5 Updated Cheat Sheet Additions

ADDITIONS TO CHEAT SHEET (Pages 17–22)

API DESIGN

REST: /register, /login, /chats, /chats/{id}/messages, /groups CRUD
 WS: SEND, TYPING, READ_ACK, DELIVERY_ACK, PRESENCE events
 Media: HTTP POST /media/upload → S3 URL → send URL via WS

CONNECTION CHOICE: WHY WEBSOCKET

HTTP → too many polls | Long Poll → reconnect storm
 SSE → server→client only, needs separate HTTP to send
 WebSocket → full-duplex, one conn for send+receive+ACK ✅

WEBSOCKET REGISTRY (Redis Hash: user_id → server + conn_id + heartbeat)
 On connect → HSET | On disconnect/TTL expiry → HDEL
 TTL = 60s | Client heartbeat every 30s to refresh
 Sticky routing: consistent hash on user_id → same ws-server always

REDIS STREAMS (hot path) vs KAFKA (analytics/CDC)
 Redis Streams: <1ms latency, real-time delivery to online users
 Kafka: durable replay, Elasticsearch indexing, audit log
 Stream key: "server:ws-server-3:inbox" (one stream per chat server)

MEDIA FLOW (2 steps)

Step 1: HTTP upload → Media Service → S3 (or pre-signed URL direct)
 Step 2: Send media_url via WebSocket SEND event (msg_type=IMAGE)

Receiver fetches from CloudFront CDN, not S3 directly

last_seen: Redis keyspace expiry event → User Management Service

→ UPDATE users SET last_seen=NOW() (PostgreSQL)

Group fanout: individual per-member (NOT broadcast) for delivery status

Offline reconnect: WS connect → fetch PENDING from Cassandra → deliver

PAGE 23 — Remaining Gaps: Scale Numbers, CAP, DB Schemas, Presence Logic

23.1 Correct Non-Functional Requirement Numbers (From Video)

Interview line: "Let me quantify the scale first — 1 billion users, 100 messages per day per user gives us 100 billion messages per day to design for."

SCALE (use these numbers in your interview — they match the video):

Total registered users: 1 billion
 Messages per user per day: 100
 Total messages per day: 100 billion (100B)
 Avg message size: ~1 KB (text + metadata combined)
 Total storage per day: 100B × 1 KB = ~100 TB/day

LATENCY TARGET:

Message delivery latency: < 300 ms (end-to-end, P99)
 (Note: 300ms is the video's target; 100ms P99 is a more aggressive production target)

AVAILABILITY vs CONSISTENCY (CAP):

Choice: HIGH AVAILABILITY + EVENTUAL CONSISTENCY

Why availability over consistency?

- This is a chat app, not a bank
- A message appearing slightly out of order is tolerable
- A user being unable to send/receive messages is NOT tolerable
- Downtime = users switch to a competitor immediately

Contrast:

- Payment system → STRONG CONSISTENCY (you cannot afford duplicate charges)
- Chat system → EVENTUAL CONSISTENCY (slight reorder is fine, downtime is not)

CAP in practice:

- Cassandra (AP): highly available, eventual consistency — perfect for messages
 - PostgreSQL (CP): consistent but can sacrifice availability — fine for user accounts
- (accounts change rarely; chat happens every second)

23.2 CAP Theorem Applied to Each Component

Component	CAP Choice	Reason
Message store (Cassandra)	AP	Cassandra — write availability is critical; slightly stale read is fine
User accounts (PostgreSQL)	CP	PostgreSQL — can't have two accounts with same phone number (strong uniqueness needed)
WebSocket registry (Redis)	AP	Redis — brief stale entry is fine; must never be unavailable (all routing breaks)
Presence service	AP	Redis — stale presence (1-2s) is

```
acceptable;
(Redis Pub/Sub) | | being unavailable means no "online" dots
Search index | AP | Elasticsearch – stale search results OK;
(Elasticsearch) | | must stay available for UX
```

Memory anchor: "Chat tolerates stale, not downtime. Banks tolerate downtime, not stale."

23.3 All Database Schemas (Explicit Field Names)

Interview line: "Let me show the schemas — these drive the query patterns and the DB choice."

User Table (PostgreSQL)

```
CREATE TABLE users (
  user_id      UUID          PRIMARY KEY,
  username     VARCHAR(100)  NOT NULL,
  email_id     VARCHAR(255),  -- for Messenger-style apps
  phone_number VARCHAR(20)    UNIQUE,    -- for WhatsApp-style apps
  status       VARCHAR(255),  -- user-set bio/status
  last_seen    TIMESTAMP,
  created_at   TIMESTAMP     DEFAULT NOW(),
  avatar_url   VARCHAR(500)   -- profile picture in S3
);

-- Query: look up by phone_number (login) → index on phone_number
-- Query: look up by user_id → primary key lookup
```

Group Table (PostgreSQL)

```
CREATE TABLE groups (
  group_id      UUID          PRIMARY KEY,
  group_name     VARCHAR(255) NOT NULL,
  description    TEXT,
  created_by     UUID          REFERENCES users(user_id),
  created_at     TIMESTAMP     DEFAULT NOW(),
  thumbnail_url  VARCHAR(500)  -- group icon in S3
);
```

Group Mapping Table (PostgreSQL) — which user belongs to which group

```
CREATE TABLE group_members (
  id            BIGSERIAL     PRIMARY KEY,  -- auto-increment surrogate key
  group_id      UUID          REFERENCES groups(group_id),
  user_id       UUID          REFERENCES users(user_id),
```

```

joined_at    TIMESTAMP    DEFAULT NOW(),
role         VARCHAR(20)  DEFAULT 'MEMBER', -- ADMIN | MEMBER
UNIQUE (group_id, user_id)
);

```

```

-- Why auto-increment id? group_id is NOT unique per row (one group, many users).
-- Surrogate key makes each row uniquely identifiable.
-- Query: get all members of a group → WHERE group_id = ?
-- Query: get all groups a user belongs to → WHERE user_id = ?
-- Both need indexes.

```

Message Table (Cassandra) — chat_id vs message_id explained

```

-- Cassandra CQL
CREATE TABLE messages (
  chat_id      TEXT,          -- conversation identifier (user_1 ↔ user_4 = chat_abc)
  timestamp    TIMESTAMP,    -- when message was sent (clustering key, DESC order)
  message_id   UUID,         -- unique ID for individual message (tie-breaker)
  sender_id    TEXT,
  receiver_id  TEXT,         -- for 1:1; for groups: the individual member being delivered to
  message      TEXT,         -- text content OR media URL (S3/CDN link)
  type         TEXT,         -- 'TEXT' | 'IMAGE' | 'VIDEO' | 'VOICE'
  delivery_status TEXT,      -- 'PENDING' | 'DELIVERED' | 'READ'
  PRIMARY KEY ((chat_id), timestamp, message_id)
) WITH CLUSTERING ORDER BY (timestamp DESC);

```

```

-- chat_id:    Identifies the CONVERSATION. User1↔User4 always = same chat_id.

```

```

--           This is the PARTITION KEY → all messages in a chat live on same node.

```

```

-- message_id: Identifies the INDIVIDUAL MESSAGE within the chat.

```

```

--           Acts as tie-breaker when two messages have identical timestamp.

```

```

-- Query: load last 50 messages:

```

```

SELECT * FROM messages WHERE chat_id = 'abc' LIMIT 50;

```

```

-- (ORDER BY timestamp DESC is built into the clustering key)

```

```

-- Query: fetch undelivered messages for a user on reconnect:

```

```

SELECT * FROM messages WHERE receiver_id = 'user4' AND delivery_status = 'PENDING'

```

```

-- (Requires secondary index on receiver_id + delivery_status, or use a separate table)

```

Why chat_id and message_id are different

chat_id = the conversation (the WhatsApp "chat" thread)

- User A ↔ User B → chat_id = "conv_AB"
- User A is in Group X → chat_id = "group_X"
- One chat_id has MANY messages over time

message_id = the individual message bubble

- "Hey how are you?" sent at 2:14pm → message_id = UUID1
- "I'm good, thanks!" sent at 2:15pm → message_id = UUID2

Analogy: chat_id = the WhatsApp thread (top-level list item)

message_id = each individual bubble inside that thread

23.4 Presence Detection Logic (Explicit)

Interview line: "Presence is trivially simple once you have the WebSocket registry — if the entry exists, the user is online. If it doesn't, they're offline."

IS USER ONLINE?

Algorithm:

HGET ws_registry user:{user_id}

→ Result not null → User is ONLINE (show green dot)

→ Result null → User is OFFLINE (show "last seen {last_seen from User DB}")

Why this works:

- Every online user has an active WebSocket → entry in ws_registry
- Entry has TTL = 60s, refreshed by heartbeat every 30s
- If user closes app or loses network → heartbeat stops → TTL expires → entry deleted
- Deleted entry = offline. It's that simple.

Presence update propagation:

When User B goes offline (registry entry expires):

1. Redis keyspace expiry event fires
2. User Management Service receives event
3. Queries last heartbeat timestamp → writes last_seen to PostgreSQL
4. Publishes PRESENCE_CHANGE event: { user_id: B, status: offline, last_seen: T }
5. All users who have an OPEN chat with B receive this event via WebSocket
6. Their UI updates: "Last seen today at 14:32"

When User B comes back online:

1. New WebSocket connection established → new ws_registry entry
2. User Management Service updates status = online
3. Publishes PRESENCE_CHANGE: { user_id: B, status: online }
4. Users who follow B see green dot appear

Performance note:

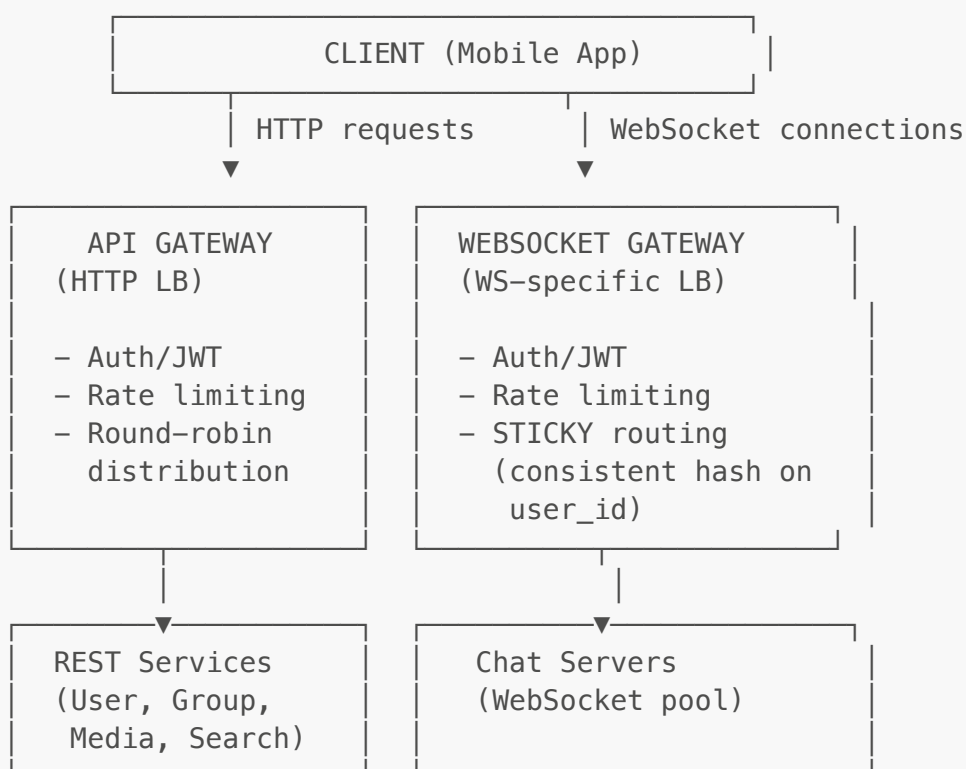
Do NOT poll ws_registry to check presence.

Use event-driven: presence changes ONLY on connect/disconnect.

At 500M users, even 1 presence poll/minute = 8.3M Redis ops/sec. Too expensive.

23.5 Two Load Balancers — Explicit Architecture

WHY TWO SEPARATE LOAD BALANCERS?



HTTP Gateway uses ROUND-ROBIN:

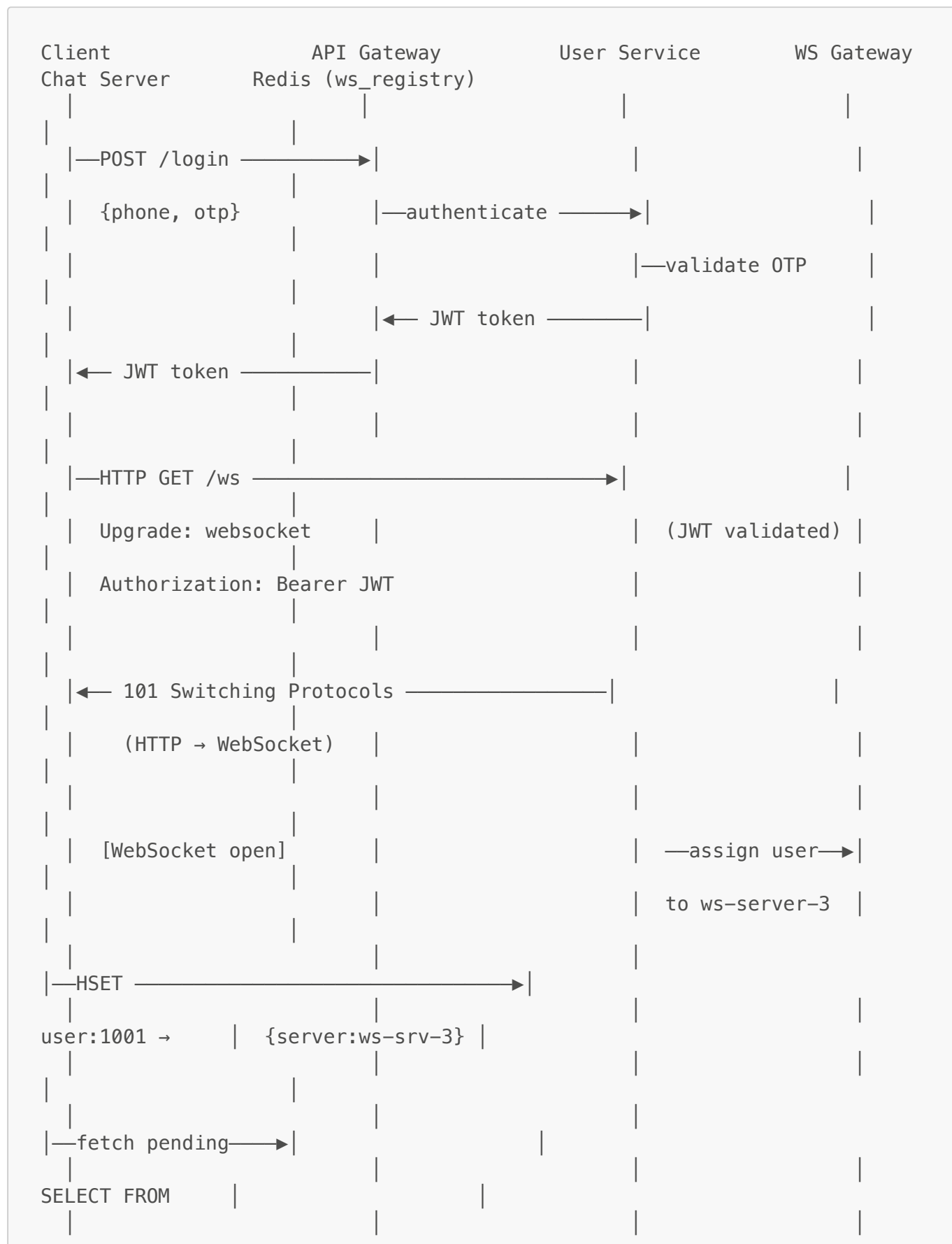
- REST calls are stateless; any server can handle any request
- Round-robin distributes load evenly

WebSocket Gateway uses CONSISTENT HASHING (sticky):

- WebSocket is stateful – tied to a specific server process
- Same user_id must always route to same chat server
- Consistent hash ring: user_id → hash → ws-server-3 (always)
- If ws-server-3 dies → ring rebalances → user reconnects → lands on new server
- Most modern LBs (Nginx, AWS ALB) support WebSocket upgrade natively
- Recommendation: dedicated WS LB to avoid HTTP/WS config conflicts at scale

Current diagrams: Component view (P7.1), ER view (P7.2), Online send-message sequence (P7.3), Media upload (P21.2), Two-LB architecture (P23.5). Added here: Login+WS handshake, Offline send flow, Group fanout, WS registry routing, Redis Streams inter-server routing.

24.1 Login + WebSocket Establishment Flow

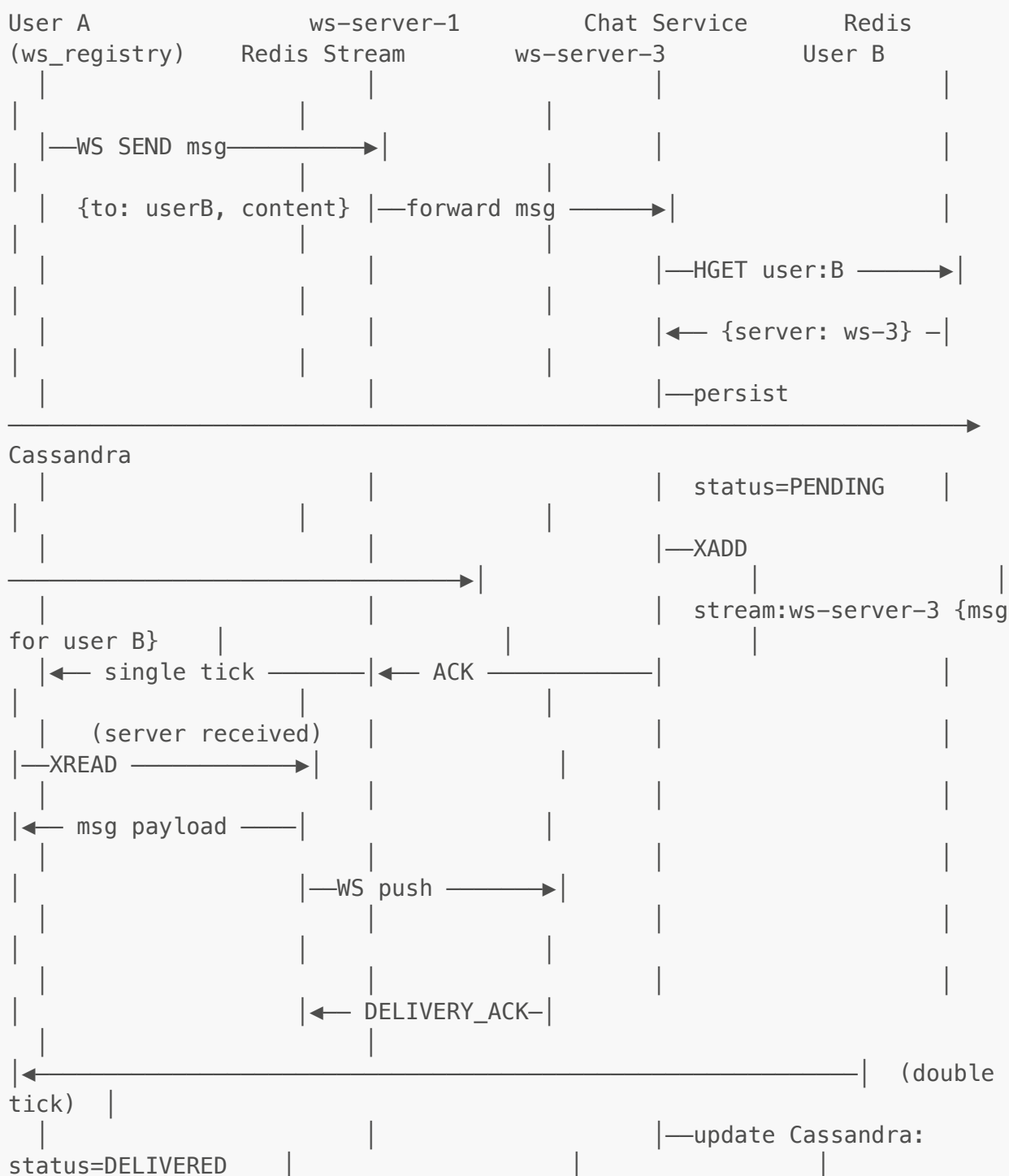


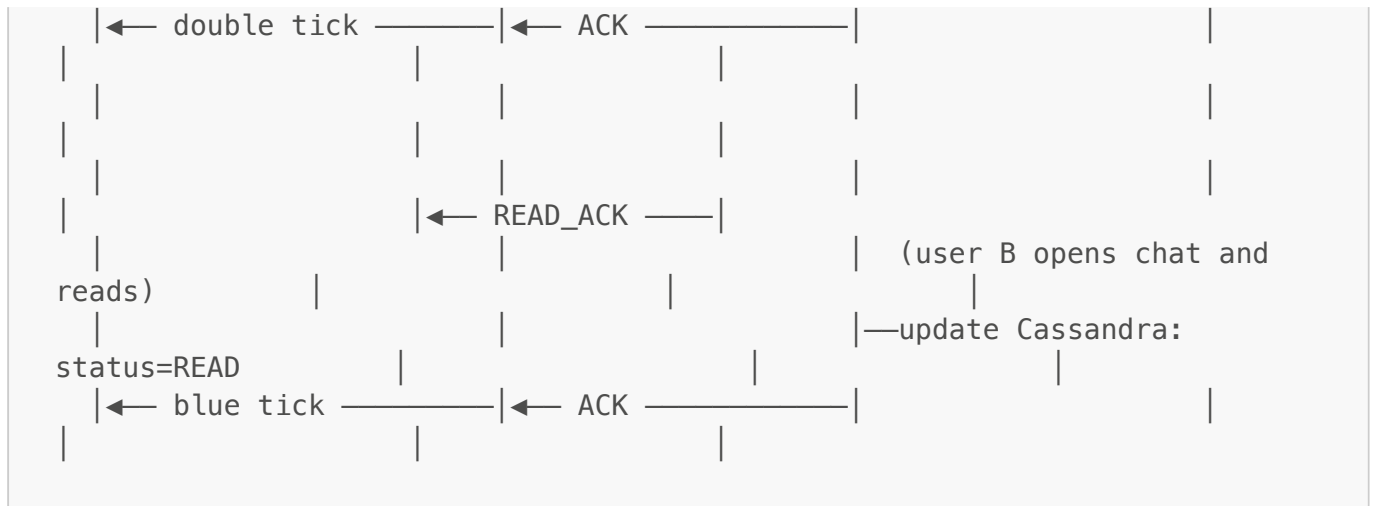
```

messages WHERE
|
| receiver=1001
|
| status=PENDING
|
| ← push pending msgs
|
| [double tick on sender's side fires]
|

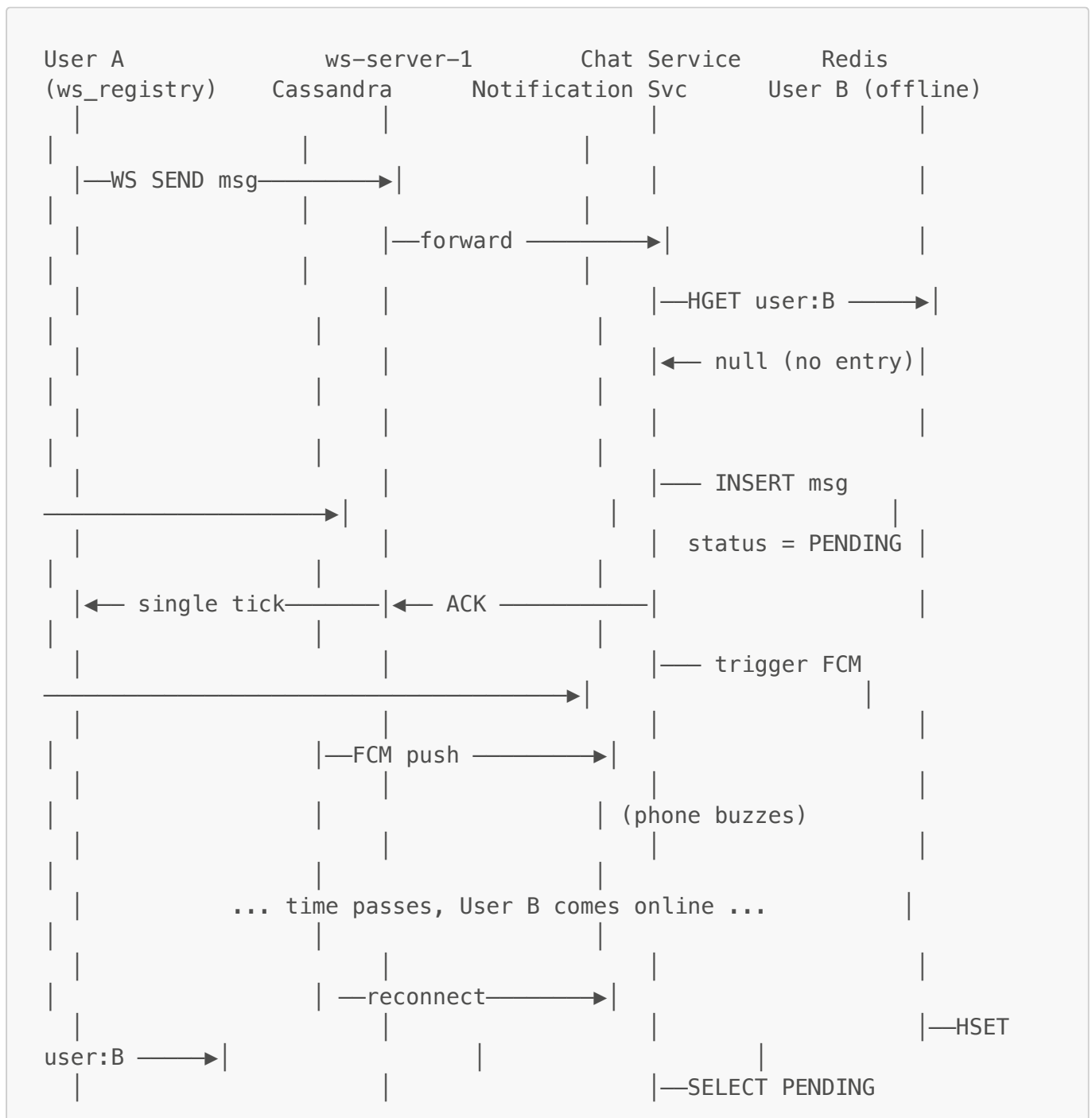
```

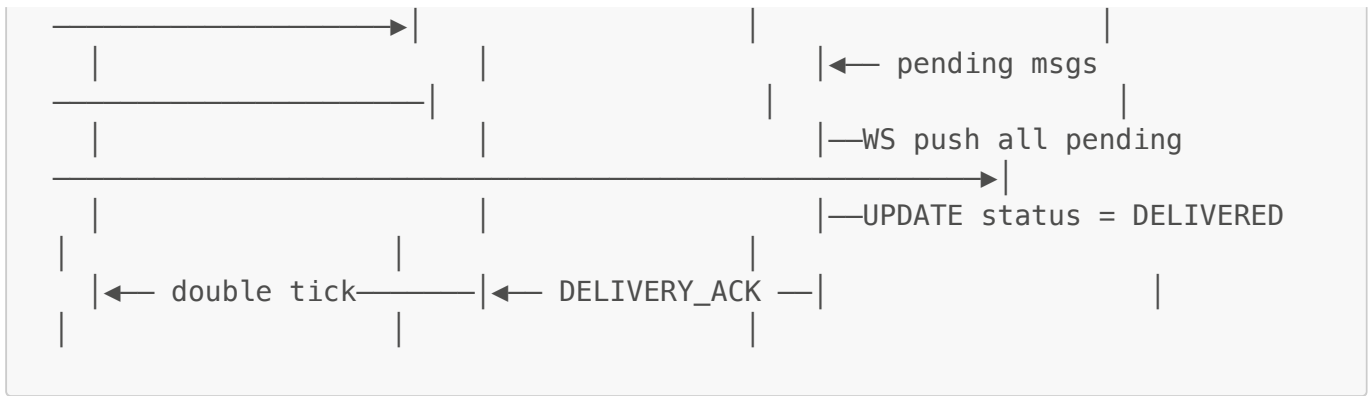
24.2 Online Message Delivery (Sender to Online Recipient, Different Servers)



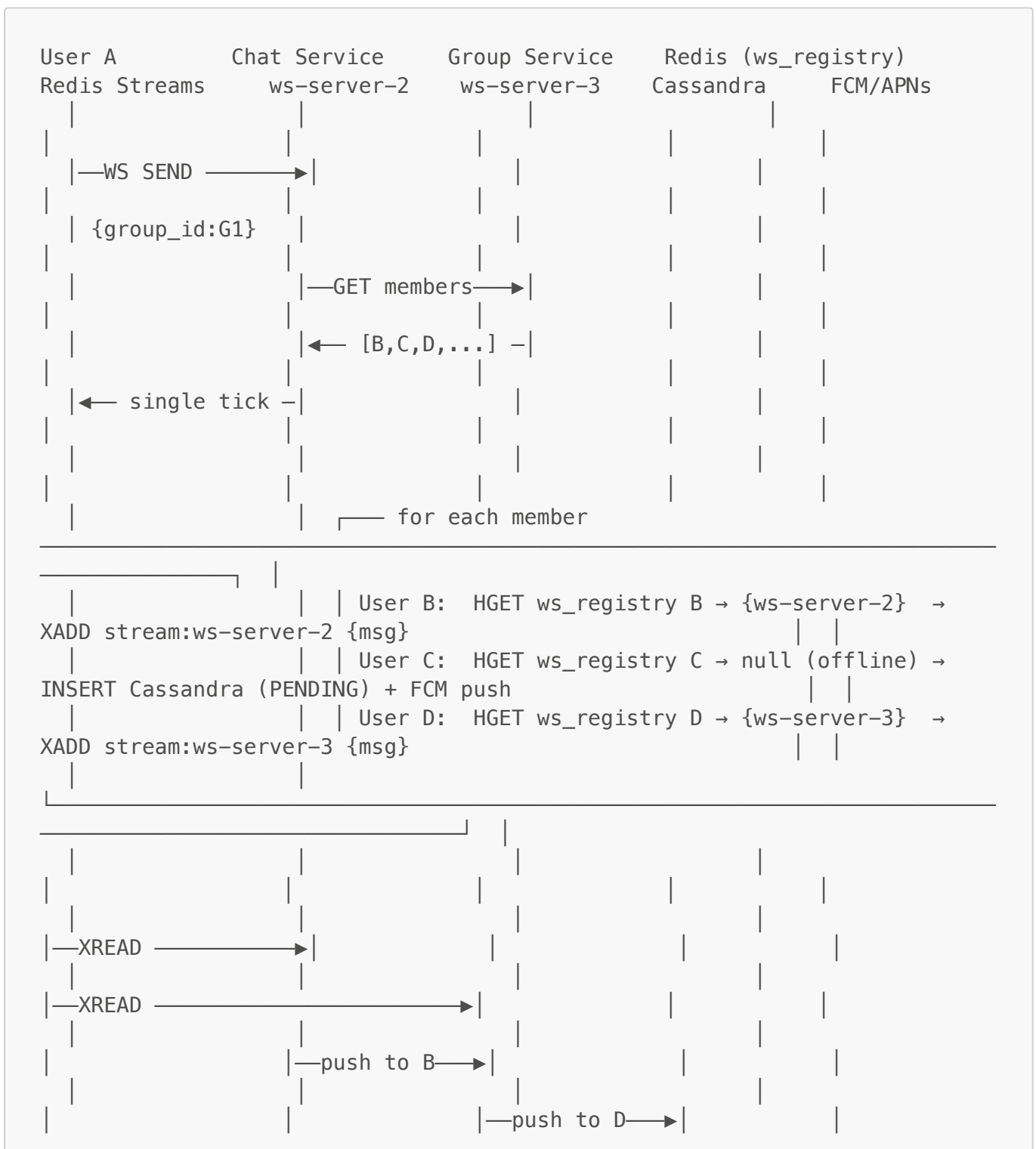


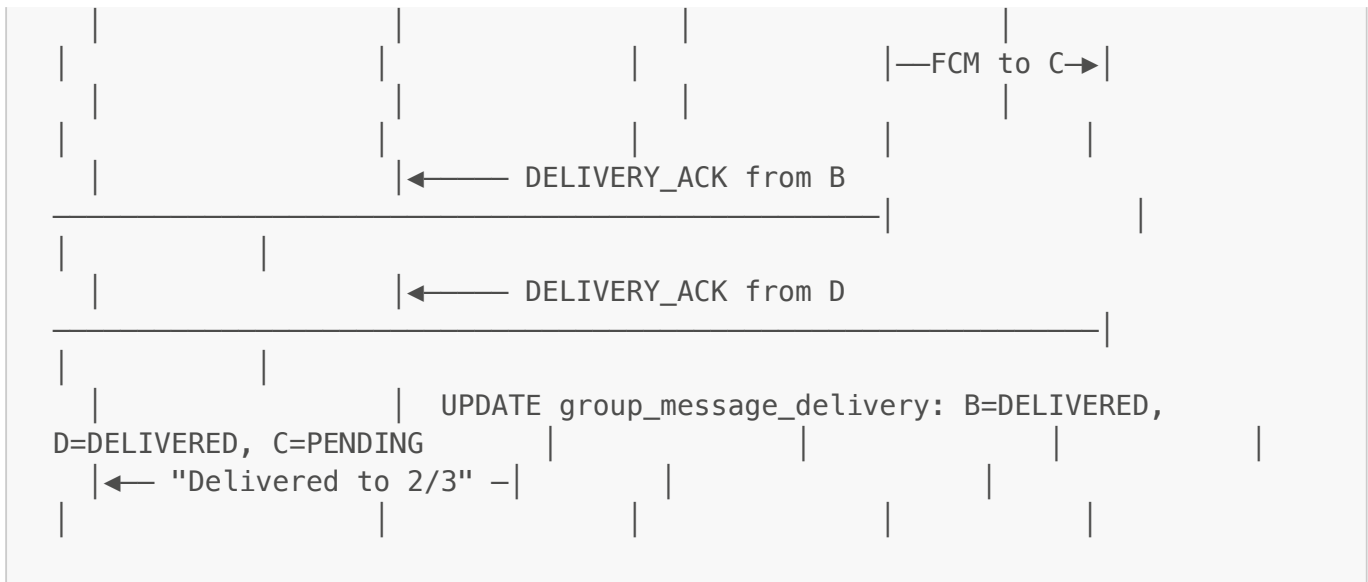
24.3 Offline Message Flow (Recipient is Offline)





24.4 Group Message Fanout Flow





24.5 Complete End-to-End Data Flow (Numbered Steps from Video)

COMPLETE DATA FLOW – WhatsApp (numbered steps)

PHASE 1: USER REGISTRATION

1. User installs app → POST /register {phone_number, name}
2. User Service creates user record in PostgreSQL
3. JWT token returned to client

PHASE 2: LOGIN + WEBSOCKET ESTABLISHMENT

4. User opens app → POST /login {phone, OTP}
5. User Service validates OTP → returns JWT
6. Client sends HTTP Upgrade request to WebSocket Gateway (with JWT)
7. WS Gateway validates JWT → upgrades to WebSocket connection
8. Chat Server registers: HSET ws_registry {user_id → server_id, conn_id}
9. Chat Server queries Cassandra for PENDING messages for this user
10. Pending messages pushed to client via WebSocket
11. Client sends DELIVERY_ACK → Cassandra updated → sender gets double tick

PHASE 3: SENDING A MESSAGE (ONLINE RECIPIENT)

12. User types message → WS SEND {chat_id, content, msg_type}
13. Chat Server receives → persists to Cassandra (status=PENDING)
14. Chat Server looks up recipient in ws_registry
15. Recipient is ONLINE → XADD to Redis Stream of recipient's server
16. Sender receives single tick (server ACK)
17. Recipient's Chat Server reads stream → pushes message via WebSocket
18. Recipient client sends DELIVERY_ACK
19. Cassandra updated to status=DELIVERED
20. Sender gets double tick

PHASE 4: SENDING A MESSAGE (OFFLINE RECIPIENT)

21. User sends message (same as step 12)
22. Chat Server: recipient NOT in ws_registry (offline)
23. Chat Server: INSERT to Cassandra (status=PENDING)
24. Sender gets single tick
25. Notification Service triggers FCM/APNs push to recipient's device
26. When recipient comes online → steps 4–11 above handle delivery

PHASE 5: SENDING MEDIA

27. User selects image → HTTP POST /media/upload to Media Service
28. Media Service uploads to S3 → returns media_url
29. Client sends WS SEND {msg_type: IMAGE, content: media_url}
30. Flow continues same as steps 13–20 above
31. Recipient's client fetches image from CloudFront CDN (not S3 directly)

PHASE 6: GROUP MESSAGE

32. User sends WS SEND {group_id, content}
33. Chat Service calls Group Service → get all member user_ids
34. For each member: check ws_registry → online or offline?
35. Online members: push via Redis Streams → their Chat Server → WebSocket
36. Offline members: persist to Cassandra + FCM push
37. Per-member delivery tracked in group_message_delivery table

PHASE 7: READ RECEIPT

38. Recipient opens chat → READ_ACK sent via WebSocket
39. Chat Server updates Cassandra: status=READ
40. Sender's Chat Server notified via Redis Streams → sender gets blue tick

PHASE 8: PRESENCE / LAST SEEN

41. WebSocket registry entry has TTL=60s, heartbeat every 30s refreshes it
42. User closes app → heartbeat stops → TTL expires after 60s
43. Redis keyspace expiry event fires
44. User Management Service: UPDATE users SET last_seen=NOW()
45. PUBLISH presence:offline → contacts with open chats see "last seen X"

24.6 Summary of All Diagrams in This Guide

Diagram	Location	What it shows
-----	-----	-----

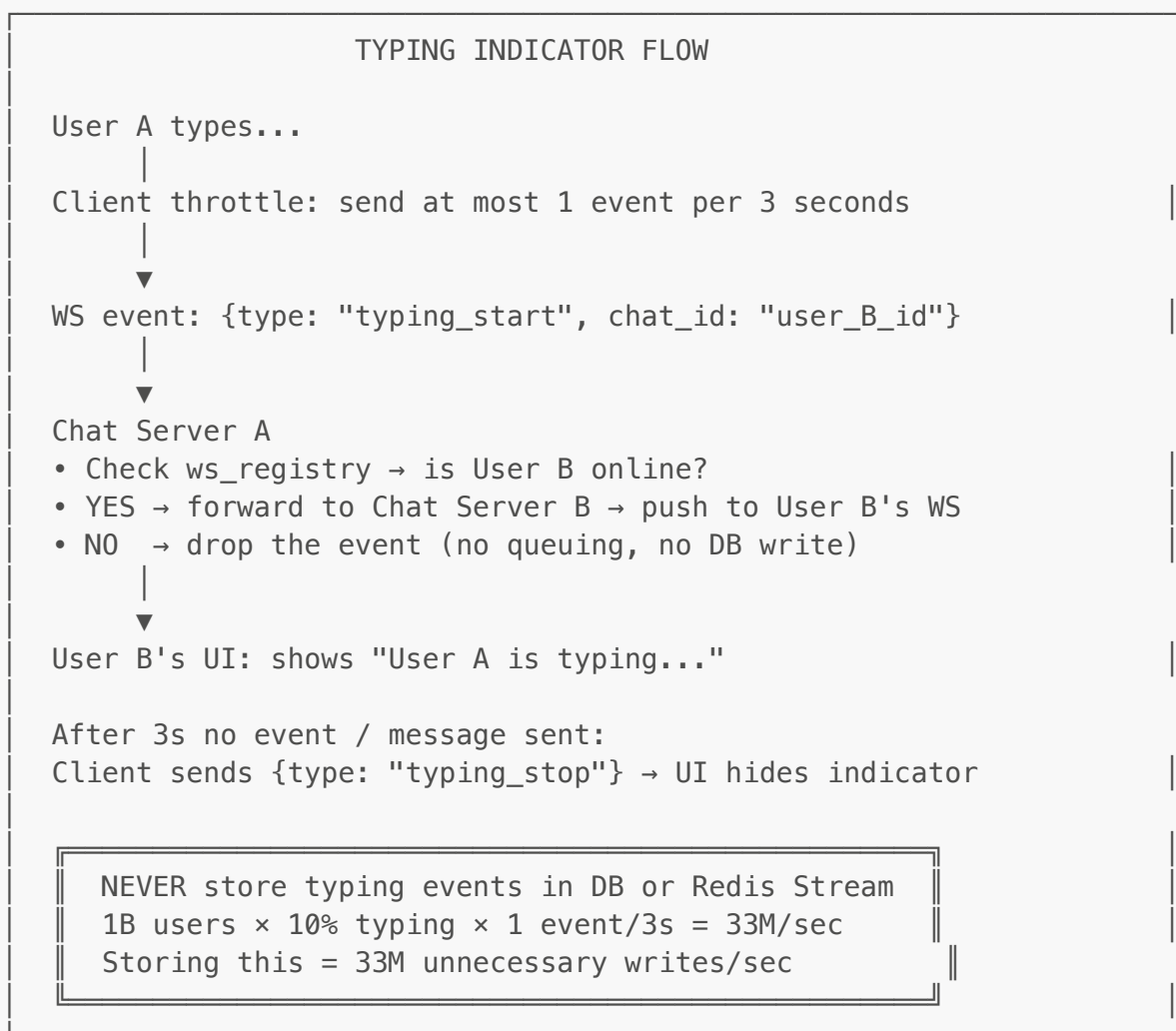
Component architecture	P7.1	All services and their connections
Entity-Relationship view	P7.2	DB tables and their relationships
Online send-message sequence	P7.3	Happy path with tick marks
Media upload (2-step)	P21.2	HTTP upload → S3 → WS URL send

Two load balancers (sticky)	P23.5	HTTP LB (round-robin) vs WS LB
Login + WS establishment	P24.1	How WebSocket gets created on login
Online msg (diff servers) Streams	P24.2	WS registry routing + Redis
Offline message flow	P24.3	Cassandra queue + FCM + reconnect
Group fanout	P24.4	Per-member online/offline split
Complete numbered data flow	P24.5	All 40+ steps end-to-end

PAGE 25 — Typing Indicators

25.1 Why Typing Indicators are Different from Messages

Typing indicators are **ephemeral** — they must never be stored in any database. They are fire-and-forget WebSocket events with client-side throttling.



25.2 Design Choices

Decision	Reason
Throttle to 1 event / 3s on client	Reduces load from keystroke-level to manageable. Typing a 50-char message = 1-2 events, not 50
Drop if recipient offline	No value in queuing a "typing" notification for later delivery
No DB write, no Redis Stream	Ephemeral by design — stale typing indicators confuse users
Group typing: broadcast to all online members	Same fire-and-forget; skip offline members entirely

25.3 Interview Trap

"Should typing events go through Redis Streams like messages?"

No. Redis Streams are for messages that must survive offline delivery. Typing indicators are only meaningful in real time. Queuing them would deliver "User A was typing" to User B hours later — nonsensical. The rule: **if it has no value when received late, don't queue it.**

PAGE 26 — Message Ordering, Delivery Guarantee & Reconnect

26.1 At-Least-Once Delivery with client_message_id

DELIVERY GUARANTEE — AT-LEAST-ONCE

PROBLEM: Network can drop the WS ACK. Client doesn't know if server got the message. Without idempotency, retry = duplicate.

SOLUTION: client_message_id (UUID generated by client before send)

Step 1: Client generates UUID before sending
 {type: "SEND", client_message_id: "abc-123", content: "Hello"}

Step 2: Server receives, checks Cassandra for client_message_id
 SELECT * FROM messages WHERE client_message_id = 'abc-123'
 • Not found → insert, return {server_message_id, status: "SENT"}
 • Found → return existing server_message_id (idempotent)

Step 3: If client gets no ACK within 5s → retry with SAME UUID
 Server detects duplicate → returns existing record → no double send

Step 4: Client maps `client_message_id` → `server_message_id` in local DB |

Delivery states:

PENDING → SENT (single tick) → DELIVERED (double tick) → READ (blue) |

Note: "At-least-once" means server may deliver twice
 Recipient client deduplicates using `server_message_id` |

26.2 Message Ordering — Server Timestamps

PROBLEM: Two users send messages simultaneously.
 Client clocks can differ by minutes. Client timestamps cause ordering chaos.

SOLUTION: Server assigns authoritative timestamp on receipt.

Client A sends M1 at `client_time=10:00:00.100`

Client B sends M2 at `client_time=10:00:00.050`

Server receives M1 first → assigns `server_ts = T=1000`

Server receives M2 second → assigns `server_ts = T=1001`

Cassandra stores both with `server_ts` as clustering key DESC

→ All clients see M1 before M2, regardless of client timestamps

For distributed servers (multiple chat server instances):

Use Lamport timestamps or snowflake IDs (not wall clock)

Snowflake = `timestamp(41b) + datacenter_id(5b) + machine_id(5b) + seq(12b)`

→ Globally unique, roughly time-ordered, no coordination needed

26.3 WebSocket Reconnect & Message Sync

RECONNECT FLOW

1. Client detects WS disconnect (network drop) |
2. Client stores `last_received_message_id` or `last_sync_timestamp` |
3. Reconnect with exponential backoff: `1s → 2s → 4s → 8s → max 16s` |
4. On new WS connection: send `last_sync_timestamp` in handshake header |

Server on reconnect:

5. Pull missed messages from Redis Stream (XREAD from beginning)
6. If stream messages > 7 days old → fallback to Cassandra:
`SELECT * FROM messages WHERE conversation_id = X
AND timestamp > last_sync_timestamp ORDER BY timestamp`
7. Deliver batch of missed messages in chronological order
8. Client deduplicates using server_message_id
9. After delivery ACK → XDEL from Redis Stream

Reconnect backoff: 1s, 2s, 4s, 8s, max 16s (exponential)
Prevents thundering herd when server restarts

26.4 Conversation ID — Partition Key Formula

The Cassandra partition key for 1-1 messages should be a **deterministic conversation_id** so both users always query the same partition:

```
conversation_id = SHA256(min(user_id_A, user_id_B) + max(user_id_A,
user_id_B))
```

Example:

```
user_id_A = "user_001"
```

```
user_id_B = "user_500"
```

```
min = "user_001", max = "user_500"
```

```
conversation_id = SHA256("user_001user_500") = "a3f9...bc12"
```

Both users query: `SELECT * FROM messages`

```
WHERE conversation_id = "a3f9...bc12"
```

```
AND timestamp < {cursor} ORDER BY timestamp DESC LIMIT 50
```

Benefits:

- Single partition → single node scan, no cross-partition query
- Deterministic → both users always land on the same partition
- No need to store two copies of each message

PAGE 27 — Disaster Recovery & Cassandra Replication

27.1 Cassandra Replication Strategy

CASSANDRA REPLICATION — RF=3

Setup: 3 replicas in primary region (us-east-1)
 Async replication to secondary region (us-west-2)

Write consistency: QUORUM (2 of 3 must acknowledge)
 → Balances durability with latency
 → If 1 node fails, writes still succeed (2 remaining nodes)

Read consistency: ONE (fastest, reads from nearest replica)
 → Eventual consistency acceptable for chat history
 → User sees their own message immediately (wrote it)

Node failure scenario:

- 1 of 3 nodes fails → QUORUM still met (2 nodes)
- 2 of 3 nodes fail → writes block (cannot reach QUORUM)
- All 3 fail → region failover

27.2 Multi-Region Failover

DISASTER RECOVERY – RTO < 5 min, RPO < 1 min

Primary region: us-east-1
 Secondary region: us-west-2 (receives async Cassandra replication)

Failover steps:

1. Primary region outage detected (health checks fail)
2. DNS failover to us-west-2 load balancer (<3 min)
3. Users reconnect WebSockets to secondary region servers
4. Message history served from replicated Cassandra
5. Redis rebuilt from Cassandra + new WebSocket connections
6. Service resumes within <5 min total

RTO (Recovery Time Objective): < 5 minutes
 RPO (Recovery Point Objective): < 1 minute (async replication lag)

Redis durability:

- AOF (Append-Only File) + RDB snapshots

- If Redis crashes: rebuild ws_registry from new connections
- Offline messages fallback to Cassandra (source of truth)

S3 media: Cross-region replication enabled

→ Media available in both regions, CloudFront serves from nearest PoP

Backups: Daily Cassandra snapshots to S3, 30-day retention

27.3 Interview Questions on Disaster Recovery

Question	Answer
"What is RF=3 in Cassandra?"	Replication Factor = 3 means each row is stored on 3 nodes. QUORUM write requires 2 of 3 to acknowledge.
"Why QUORUM writes but ONE reads?"	QUORUM writes = durability (2 nodes have the data before returning success). ONE reads = speed (eventual consistency OK for chat).
"What is RTO vs RPO?"	RTO = how long to restore service after failure. RPO = how much data you can afford to lose (lag between primary and replica).
"How do you handle the Redis state after region failover?"	ws_registry is rebuilt naturally as users reconnect. Offline message queue in Redis Stream is lost, but those messages exist in Cassandra — served on reconnect.

PAGE 28 — Message Search with Elasticsearch

28.1 Architecture

MESSAGE SEARCH — ELASTICSEARCH

Write path (async CDC):

Cassandra → CDC consumer (Kafka/Debezium) → Elasticsearch indexer

Elasticsearch index: messages-{yyyy-MM-dd}

Fields:

message_id — keyword (exact match)
 sender_id — keyword
 receiver_id — keyword
 group_id — keyword (null for 1-1 messages)

```

content      - text (analyzed, full-text search)
type         - keyword (text/image/video)
timestamp    - date

```

```

Search query: GET /v1/messages/search?q=hello&chat_id={user_id}
Elasticsearch query: match on content, filter by chat_id,
sorted by timestamp DESC, returns highlights

```

```

Optimization: Index only last 6 months of messages
→ Older messages require direct Cassandra scan (slower but rare)

```

28.2 Why Not Search Cassandra Directly?

Cassandra	Elasticsearch
Optimized for partition-key queries	Optimized for full-text search
No full-text index	Inverted index on content field
ALLOW FILTERING = full scan, catastrophic at scale	Search across all messages in < 100ms
Cannot rank by relevance	BM25 relevance scoring, highlights

28.3 Interview Trap

"Why not use Cassandra's LIKE queries for search?"

LIKE '%hello%' requires **ALLOW FILTERING** in Cassandra — a full table scan across all partitions. At 100B messages/day this is unusable. Elasticsearch's inverted index answers "which documents contain 'hello'" in $O(\log n)$ time.

PAGE 29 — Group Schema Detail (Group Mapping Table)

29.1 Two-Table Group Schema

The **blogs_notes** and **chatapps.md** sources explicitly use a **separate join table** for group membership rather than storing member_ids as an array:

Table 1: groups (PostgreSQL)

```

group_id      uuid PRIMARY KEY
group_name    varchar(255)
group_pic_url varchar(500)
created_by    uuid FK → users(user_id)
description   text
created_at    timestamp

```

```
updated_at    timestamp
```

Table 2: group_members (PostgreSQL) – the join table

id	bigserial PRIMARY KEY	← autoincrement (not group_id alone, because group has many users)
group_id	uuid FK → groups	
user_id	uuid FK → users	
role	varchar(20)	← 'ADMIN' or 'MEMBER'
joined_at	timestamp	

Why a separate join table, not a `member_ids uuid[]` array in the groups table?

Array column	Separate join table
Adding a member = UPDATE the array (lock on the row)	Adding a member = INSERT one row (no contention)
Querying "which groups is user X in?" = full table scan	<code>SELECT group_id FROM group_members WHERE user_id = X</code> = index scan
No per-member metadata (role, join date)	Per-member role, join date, removed_at easily stored
Atomic add/remove for large groups is tricky	Each membership is a first-class row with its own lifecycle

29.2 Group Read Receipts in Cassandra

For group messages, per-member read status is tracked:

```
group_messages table (Cassandra):
partition_key  = group_id
clustering_key = timestamp DESC
message_id     uuid
sender_id      uuid
content        text
type           text
media_url      text
read_by        set<uuid> ← user_ids who have read this message

Query: "Who has read message X?"
      SELECT read_by FROM group_messages WHERE group_id = X AND message_id = Y

When member reads: UPDATE group_messages SET read_by = read_by + {user_id}
                    WHERE group_id = X AND message_id = Y

UI shows: "Read by 3 of 5 members"
```

30.1 Key Numbers (from blogs_notes)

SCALING NUMBERS TO REMEMBER	
WS connections per instance	100,000 concurrent WebSocket connections per gateway instance
Gateway instances for 10M concurrent users	~100 instances (10M / 100K per instance)
Messages/sec per gateway	10,000 messages/sec per instance
Reconnect backoff	1s → 2s → 4s → 8s → max 16s (exponential, prevents thundering herd)
Heartbeat interval	30s ping; 60s TTL on Redis key (2 missed heartbeats = offline)
Typing indicator throttle	Max 1 event per 3 seconds from client
Redis Stream offline window	7 days; fallback to Cassandra after
Cassandra TTL for messages	Optional: 2 years auto-delete
Media presigned URL expiry	15 minutes
Group max size	256 members (WhatsApp production limit)
Cassandra replication	RF=3, QUORUM writes (2 of 3), ONE reads
Uptime SLA	99.99% = 52 minutes downtime per year
Message delivery latency end-to-end calculation	Typical: 55ms (10ms WS + 30ms Cassandra + 5ms Redis lookup + 10ms WS deliver)

30.2 Compression and Bandwidth Optimization

Technique: zlib compression on WebSocket frames

Compression ratio: 3:1 (text messages compress very well)

Savings: 66% bandwidth reduction for mobile users

Implementation: WebSocket permessage-deflate extension (RFC 7692)

When to mention: "At 100B messages/day × avg 1KB = 100TB/day.

With 3:1 compression → ~33TB/day transferred, saving 67TB/day."

Technique: Message batching (group sends)

Multiple messages in a single WebSocket frame

Reduces network overhead by ~80% for burst sends

Useful for: reconnect sync (deliver 50 missed messages in one frame)

Technique: Client-side SQLite cache

Store last 1000 messages per chat on device

App open = instant load from local cache, background sync from server

Avoids a Cassandra round-trip for every chat open

Dedup using message_id on sync

30.3 Updated Diagrams Index

Diagram	Location	What it shows
-----	-----	-----

Component architecture	P7.1	All services and their connections
Entity-Relationship view	P7.2	DB tables and their relationships
Online send-message sequence	P7.3	Happy path with tick marks
Media upload (2-step)	P21.2	HTTP upload → S3 → WS URL send
Two load balancers (sticky)	P23.5	HTTP LB (round-robin) vs WS LB
Login + WS establishment	P24.1	How WebSocket gets created on login
Online msg (diff servers) Streams	P24.2	WS registry routing + Redis
Offline message flow	P24.3	Cassandra queue + FCM + reconnect
Group fanout	P24.4	Per-member online/offline split
Complete numbered data flow	P24.5	All 40+ steps end-to-end
Typing indicator flow lifecycle	P25.1	Fire-and-forget WS event
At-least-once delivery	P26.1	client_message_id dedup chain
WebSocket reconnect flow	P26.3	Backoff + sync from Redis/Cassandra
Cassandra RF=3 replication	P27.1	QUORUM writes, ONE reads
Multi-region failover	P27.2	DNS failover, RTO/RPO
Elasticsearch search path	P28.1	CDC → ES index → search query
NLB vs ALB trap	P31.1	Layer 4 TCP pass-through vs Layer 7 terminate
Active Watchers pattern (SADD/SREM)	P31.2	Presence fan-out optimization
Senior Trap Q&A	P31.3	Race conditions, E2E encryption, celebrity scale

PAGE 31 — Senior Interview Traps (15 YOE Level)

31.1 Layer 4 NLB vs Layer 7 ALB — The #1 WebSocket Trap

x WRONG: "I'll put an Application Load Balancer (ALB) in front of my WebSocket servers."

WHY IT'S WRONG:

ALB = Layer 7 (HTTP). It TERMINATES the TCP connection at the LB, creates a NEW TCP connection to the backend server.

WebSocket upgrade handshake happens on the original TCP connection. When ALB terminates that TCP → WebSocket upgrade FAILS.

✓ CORRECT: Use Network Load Balancer (NLB) – Layer 4 (TCP).

NLB passes raw TCP packets THROUGH to the backend server. The WebSocket HTTP upgrade completes on the actual server. NLB uses consistent hashing on client IP → sticky sessions work.

Summary:

ALB (Layer 7)	Terminates TCP. Understands HTTP. Breaks WS. Use for: REST APIs, HTTP microservices.
NLB (Layer 4)	Passes raw TCP through. Invisible to WS. Use for: WebSocket, gRPC, any TCP protocol.

In WhatsApp architecture: TWO LBs in parallel

- API Gateway (ALB) → handles REST (register, history, group CRUD)
- NLB → routes WebSocket connections to Chat Servers

31.2 Active Watchers Pattern — Presence Fan-Out Optimization

PROBLEM:

500M DAU × heartbeat every 15s = 33M presence updates/sec.

Average user has 200+ contacts.

Naive approach: notify ALL contacts when user goes online/offline.

200 contacts × 33M events/sec = 6.6B fan-out operations/sec. UNUSABLE.

SOLUTION: Active Watchers

Only notify users who currently have this user's chat OPEN.
 At any moment, a user has maybe 1–5 active open chats.
 Fan-out drops from O(all contacts) to O(active watchers).

REDIS IMPLEMENTATION:

Alice opens Bob's chat:

```
SADD presence:watchers:{bob_id} {alice_ws_server_id}
```

Alice closes Bob's chat / backgrounds app:

```
SREM presence:watchers:{bob_id} {alice_ws_server_id}
```

Bob goes offline (Redis TTL expires or graceful disconnect):

```
SMEMBERS presence:watchers:{bob_id}
```

→ returns [ws-server-3, ws-server-7]

→ notify ws-server-3 and ws-server-7

→ those servers push to Alice (and others on those servers):

```
"Bob: last seen 10:05 AM"
```

Contacts list "last seen":

NOT pushed proactively. Fetched ON DEMAND when user opens contacts tab.

GET user:{id}:last_seen → calculate "Last seen 2 hours ago"

Celebrity case (10M followers):

Maybe 10,000 users have celebrity's chat open at any time.

SMEMBERS returns 10,000 watchers → 10,000 server pushes.

vs. naive 10M notifications. 1000× reduction.

31.3 Senior Trap Q&A — 8 Questions with Full Answers

CATEGORY 1 – RACE CONDITIONS

Q: "Alice sends message ABC. Server crashes AFTER writing to Cassandra but BEFORE sending ACK to Alice. Alice retries. Bob gets it twice. How do you prevent duplicate delivery?"

A: Client-generated idempotent UUID.

Alice generates message UUID on device BEFORE sending.

Every retry sends the SAME UUID ("abc-uuid-123").

Server receives retry:

1. SELECT * FROM messages WHERE message_id = 'abc-uuid-123'

2. EXISTS → send ACK again (idempotent), skip re-processing

3. NOT EXISTS → first time, insert and process normally

UUID indexed in Cassandra → O(1) lookup, no performance hit.

Pattern: at-least-once delivery (Redis Streams/Cassandra) + client dedup
 = exactly-once UX.

Q: "User logs in on iPhone and Android simultaneously. Both register on different WS servers. How do you handle dual-device sessions?"

A: Last-writer-wins in connection registry.
iPhone connects: HSET user:{id}:server "ws-1"
Android connects: HSET user:{id}:server "ws-7" ← overwrites
All new messages now route to ws-7 (Android).
iPhone session on ws-1 still exists in its local map.
Fix: on register, send "session displaced" event to old server.
WS-1 terminates iPhone session with message:
"WhatsApp is active on another device."
← This is exactly WhatsApp's real behavior.
Multi-device (WhatsApp's newer model): encrypt message separately
for EACH linked device → N encrypted copies per send, N deliveries.

CATEGORY 2 – FAILURE MODES

Q: "A WS server crashes. 100K users lose their connections.
What happens to in-flight messages?"

A: Three safety layers – no messages lost.

1. Kafka durability: messages already published survive the crash.
Kafka replication factor=3. Another consumer picks up the event.
2. Cassandra: pending_messages written BEFORE ACK was sent to sender.
When users reconnect to any healthy WS server → drain Cassandra.
3. Client retry: no ACK in 30s → exponential backoff resend.
The 100K displaced users reconnect within seconds (mobile auto-reconnect).

Brief delay (seconds) for in-flight messages. Zero message loss.

Q: "Alice sends msg1, msg2 rapidly. Network reorders them.
Bob sees msg2 before msg1. How do you guarantee ordering?"

A: Server-assigned Snowflake IDs, never client timestamps.
msg1 arrives at server first → assigned snowflakeId = 1000
msg2 arrives second → assigned snowflakeId = 1001
Even if network delivers msg2 first to Bob's device:
Bob's client buffers 100ms and sorts by snowflakeId before display.
Cassandra clustering key = snowflakeId → stored in chronological order.
Client clocks are NEVER used for ordering – device clocks can be
wrong, manually set ahead, or skewed across timezones.

CATEGORY 3 – E2E ENCRYPTION (Signal Protocol)

Q: "How does Alice send an encrypted message to Bob who is OFFLINE?
She can't do a live key exchange if Bob isn't connected."

A: X3DH (Extended Triple Diffie-Hellman) – offline key exchange.

When Bob REGISTERS, he uploads to server:

- Identity Key (permanent public key – never changes)
- Signed Pre-Key (rotated monthly)
- One-Time Pre-Keys (pool of 100 single-use keys, replenished as used)

When Alice wants to message Bob (Bob offline):

1. Alice fetches Bob's public keys from server
2. Alice runs X3DH math locally → derives shared secret (4 Diffie-Hellman operations; no coordination with Bob needed)
3. Alice encrypts message with derived key
4. Server stores encrypted blob (cannot read it – has no private keys)
5. Bob comes online → fetches encrypted blob
6. Bob runs same X3DH math → derives same shared secret
7. Bob decrypts locally on device

Server = blind relay. It stores ciphertext it physically cannot decrypt.
Private keys NEVER leave the device.

Q: "Attacker steals Bob's device and gets all private keys.

Can they decrypt all of Bob's past messages?"

A: No – Double Ratchet provides forward secrecy.

Each message derives a NEW key from the previous via one-way KDF:

$\text{Key}_1 \rightarrow \text{KDF} \rightarrow \text{Key}_2 \rightarrow \text{KDF} \rightarrow \text{Key}_3 \rightarrow \text{KDF} \rightarrow \text{Key}_4 \dots$

If attacker gets Key_3 (message 3's key):

- x Cannot decrypt messages 1 and 2 (KDF is one-way, can't reverse)
- x Cannot decrypt messages 4+ (need the next ratchet step, which they don't have because it's derived interactively)

Each message is protected by a key that exists only for that message.

Forward secrecy = past messages safe even after device compromise.

This is why WhatsApp itself cannot read your chat history.

CATEGORY 4 – SCALE LIMITS

Q: "A celebrity has 10 million followers. When they go online, do we send presence updates to all 10M?"

A: No – Active Watchers only (see PAGE 31.2).

At any moment, ~10,000 users have the celebrity's chat open.

Only those watchers get the presence push.

10,000 vs 10,000,000 = 1000× reduction in fan-out.

Contacts list last_seen: fetched on-demand when contacts tab is opened.

Never proactively pushed to all followers.

Q: "WhatsApp has 2B users. How many WS servers do you need?"

A: Capacity math:

2B users, ~25% online concurrently = 500M concurrent WS connections.

Per server: 100K connections (standard commodity server).

Servers needed: $500\text{M} / 100\text{K} = 5,000$ WS servers.

With Netty + JDK 21 virtual threads: 500K–1M connections per server.
→ 500 to 1,000 servers at maximum efficiency.
WhatsApp historically used Erlang/BEAM which achieves similar density.
Key numbers to remember:
100K connections/server (conservative)
500K connections/server (Netty + virtual threads)
33M presence writes/sec (500M users × 1 heartbeat/15s)
4 PB/day media storage (average WhatsApp media volume)
2,500 WS servers at 200K connections each (reasonable interview answer)

31.4 What NOT to Say — Additional Traps

- x "I'll use an Application Load Balancer (ALB) for WebSocket"
→ ALB is Layer 7 and terminates TCP. WebSocket upgrade fails at ALB.
Correct answer: Layer 4 NLB passes raw TCP through to the server.
- x "WS servers call each other directly to route messages"
→ $O(N^2)$ connections. 2,500 servers × 2,499 = 6.25M peer connections.
Use Kafka as a hub: $O(N)$ – every server only connects to Kafka.
- x "TLS encryption satisfies the WhatsApp encryption requirement"
→ TLS = encryption in transit only. Server decrypts at rest.
WhatsApp requires E2E encryption (Signal Protocol).
Server stores only ciphertext it CANNOT read.
TLS and E2E encryption are completely different security properties.
- x "I'll put presence (online/offline) in PostgreSQL/MySQL"
→ 33M writes/sec destroys any RDBMS. Redis TTL is the answer.
Presence is ephemeral – auto-expiry on disconnect, sub-ms reads.
- x "Notify all contacts when a user goes online"
→ For a celebrity: 10M notifications per login event = DDOS of your own system.
Use Active Watchers: only notify users who have the chat open.

Guide version: 5.0 | Pages 25-31 added | Topic: Chat Application System Design like WhatsApp |
Audience: 5–15 YOE Java Fullstack